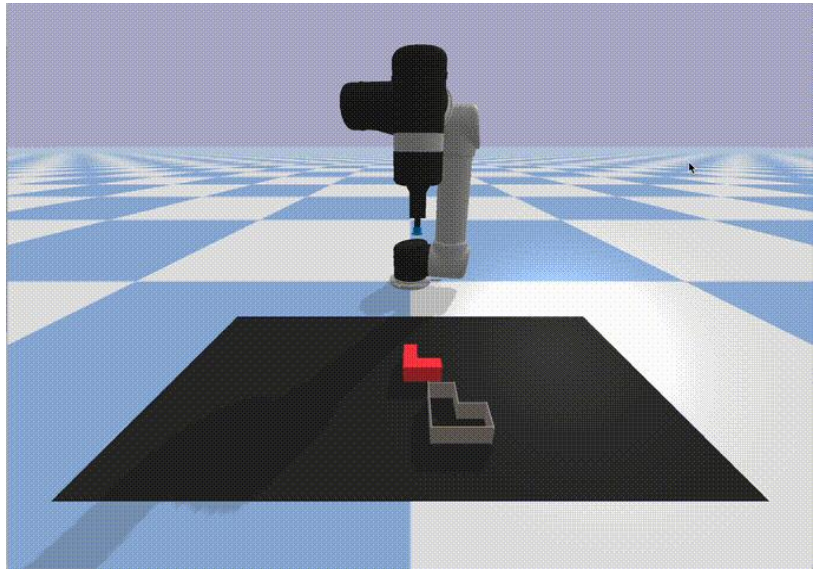
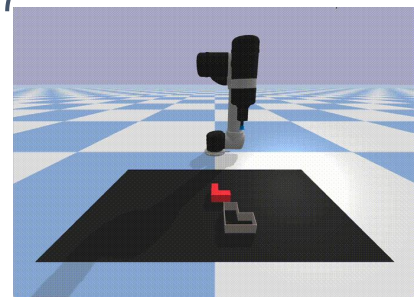
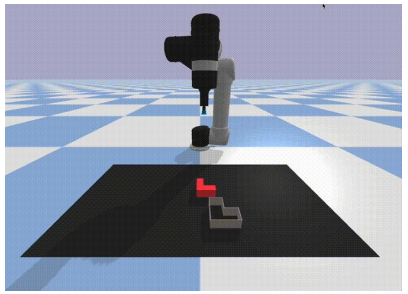
Robotic Manipulation

Robot Manipulation Task: block insertion

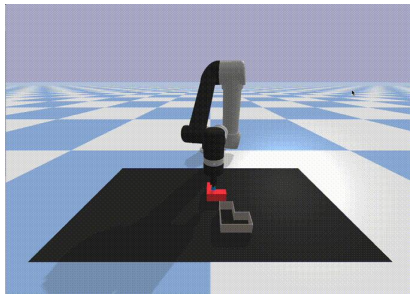




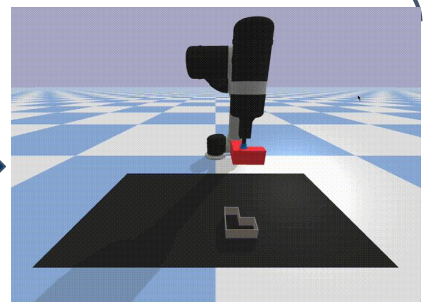
To prepick pose



To pick pose

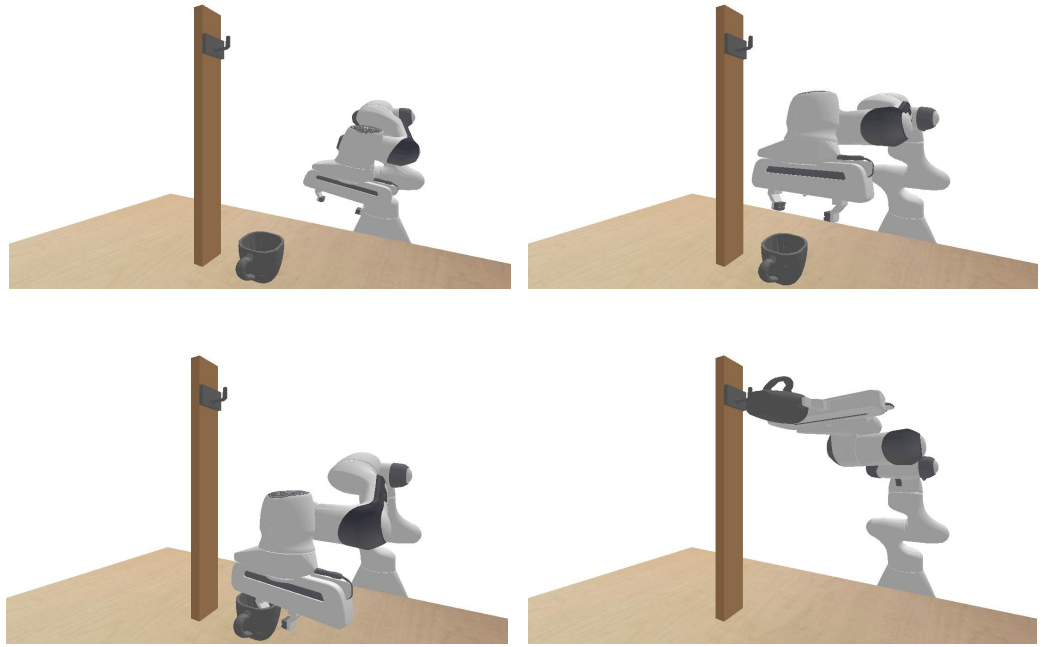


To preplace pose



To place pose

The Concept Map for Robot Manipulation



- Forward Kinematics
- 6D Pose Estimation
- Inverse Kinematics
- Grasping

Homework 4

- Forward Kinematics
  - 4 D-H parameters **p. 23** (Task1)
  - D-H conventions **p. 36 and 37** (Task1)
- Differential Kinematics
  - Geometric Jacobian **p. 57** (Task1)
- Inverse Kinematics
  - Iterative inverse kinematics **p. 45** (Task2)
  - Bonus: other methods you like (Task2)

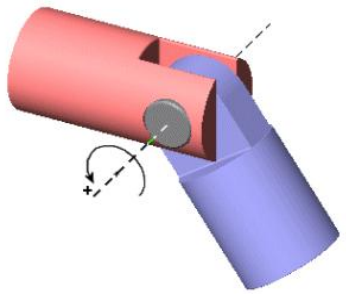
Robot Manipulator

- A robot arm (aka serial link manipulator) consists of series of rigid links, connected by joints (motors)
  - Joints provide relative motion
  - Links are rigid members between joints
- Each joint provides a degree-of-freedom (DoF)
- Most robot arms have 4 to 7 DoF

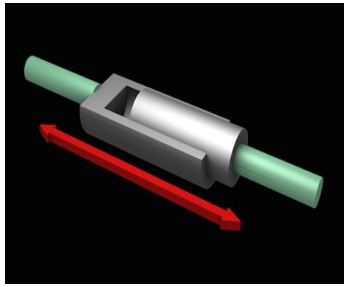
Types of Joint

all of the higher degree-of-freedom joints can be described by combinations of one degree-of-freedom joints

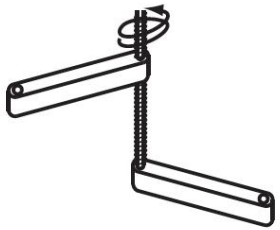
- Revolute joint
  - Single DoF
  - Rotate about an axis
- Prismatic joint
  - Single DoF
  - Translation along an axis
- Helical joint
  - Also called screw joint
  - Single DoF
  - Simultaneous rotation and translation about the screw axis



Revolute Joint



Prismatic Joint

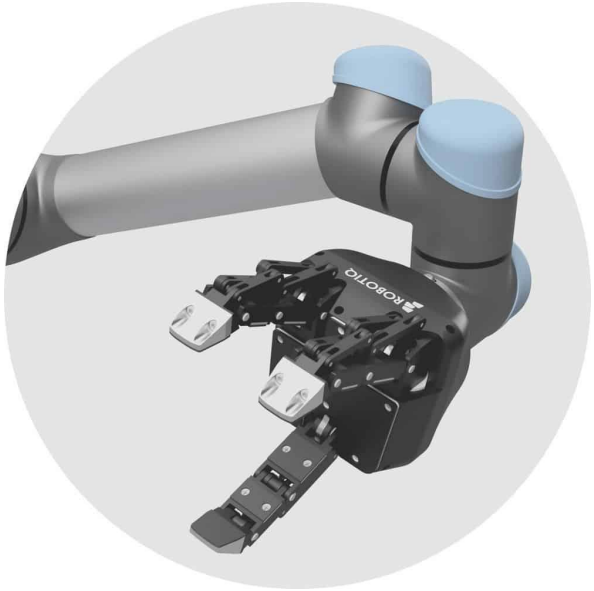


Helical Joint

End-effector

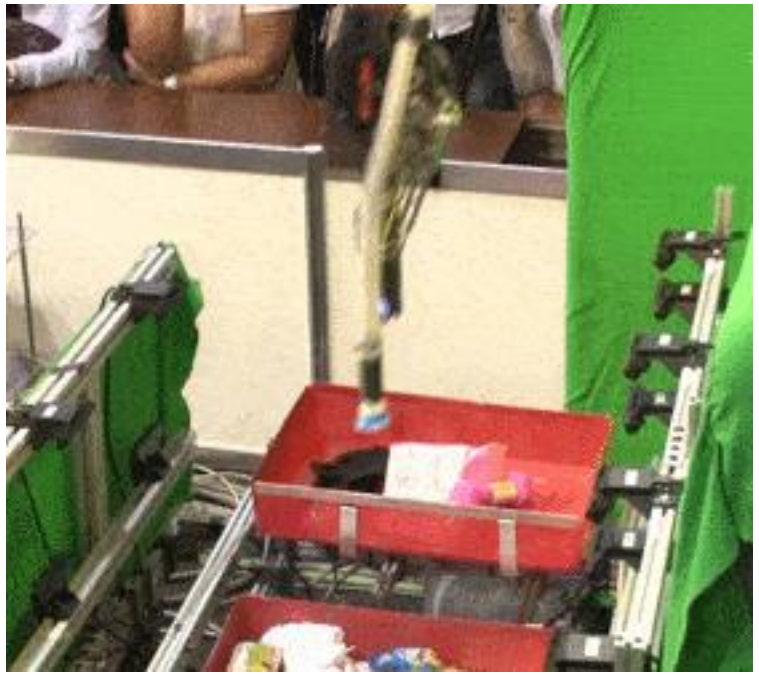
- A device or tool connected to the end of a robot that enables to perform a specific task
- Types
  - Grippers
    - Grasp and manipulate objects
  - Tools
    - to perform a process
    - Spray painting, cutting or 3D print

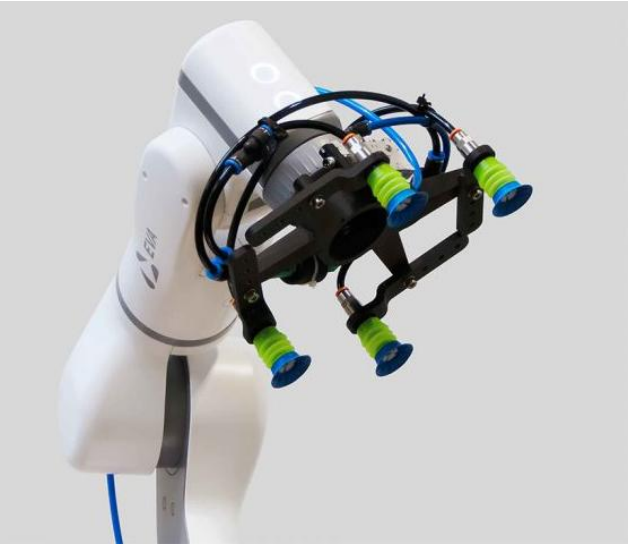
End-effector

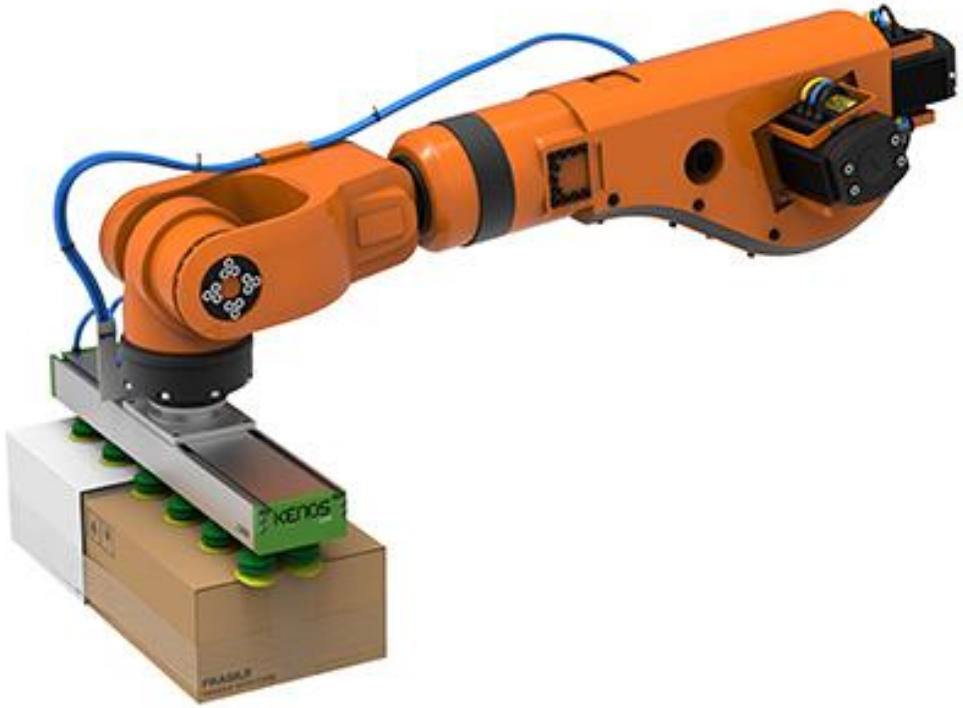


Rigid gripper with different complexity

End-effector: Grippers







Suction gripper

https://transporternets.github.io/

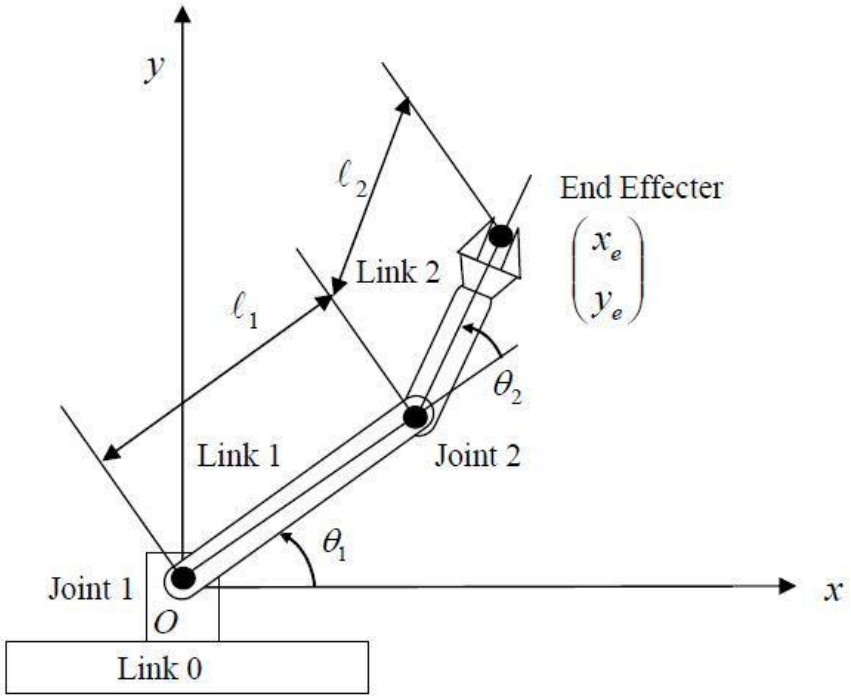
How the Tesla Model S is Made?

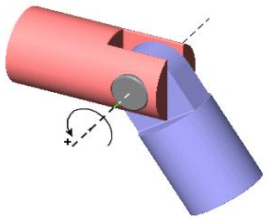
https://youtu.be/8_lfxPI5ObM

Serial Link Arms

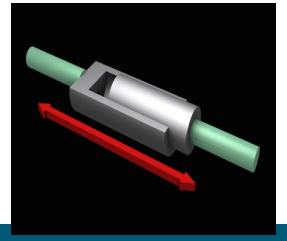
- Number the links in sequence
- For a robot with n joints:
  - Base is Link 0.
  - End-effector (tool) is attached to Link n.
  - Joint i connects Link $i-1$ to Link i
  - We define the joint variable q_i for joint i as:

$$q_i = \begin{cases} \theta_i \text{ if joint } i \text{ is revolute} \\ d_i \text{ if joint } i \text{ is prismatic} \end{cases}$$



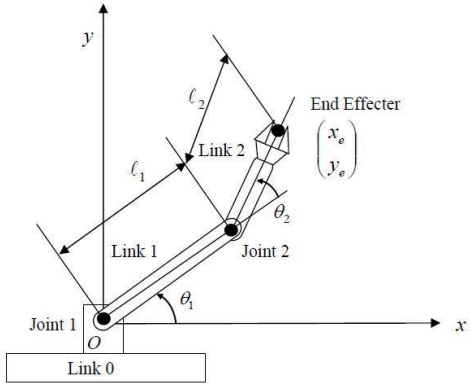


Revolute Joint



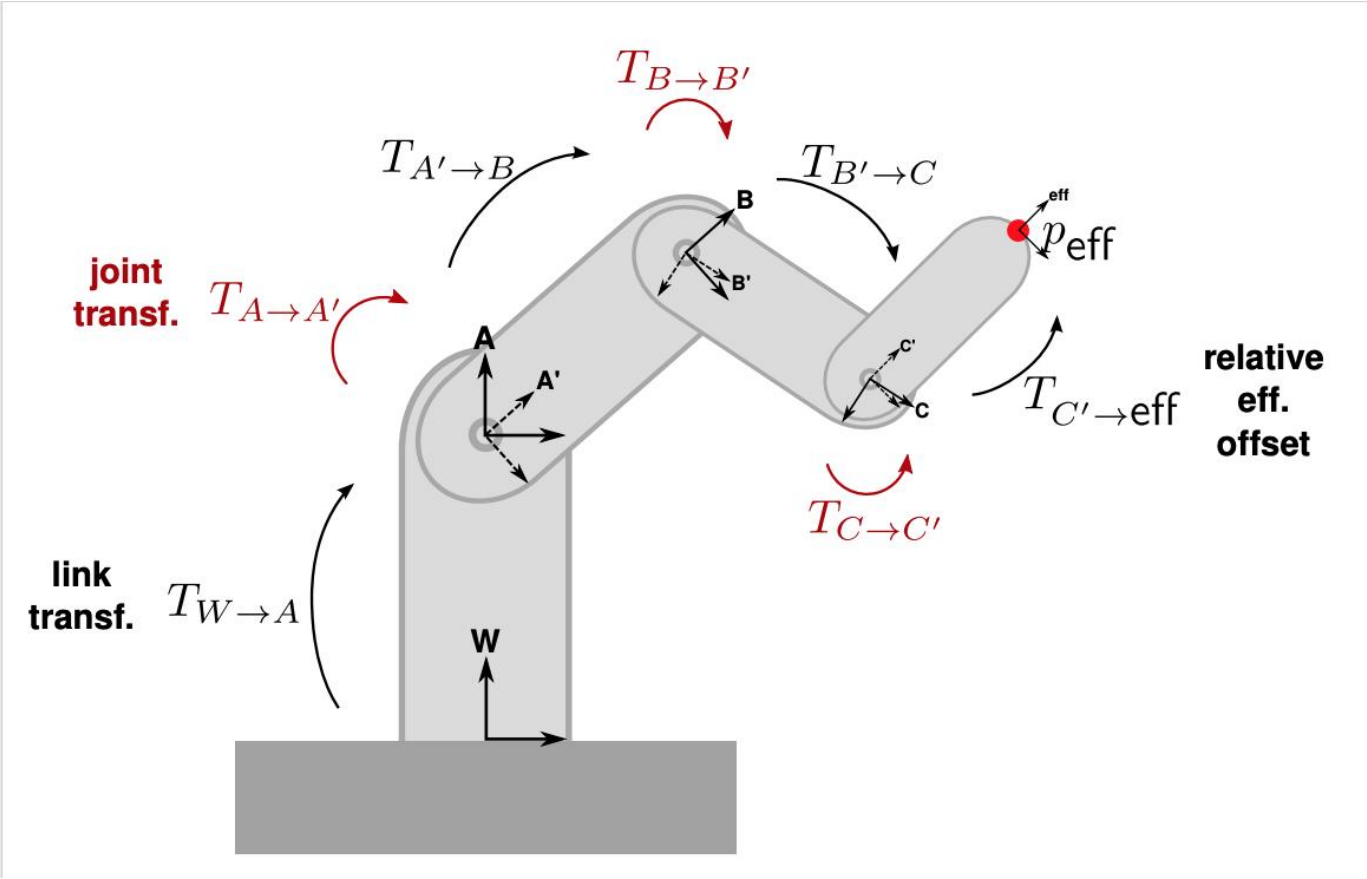
Prismatic Joint

Manipulator Kinematics



- Kinematics describes the position and motion of a robot, without considering the forces required to cause the motion
- Forward Kinematics:
  - Given the value for each joint variable, determine the position and orientation of the end-effector (gripper, tool) frame
- Each link is a rigid body
  - Describe the position and orientation of a rigid body
  - Attach a coordinate frame to the body.
  - Specify the position and orientation of the coordinate frame relative to some reference frame

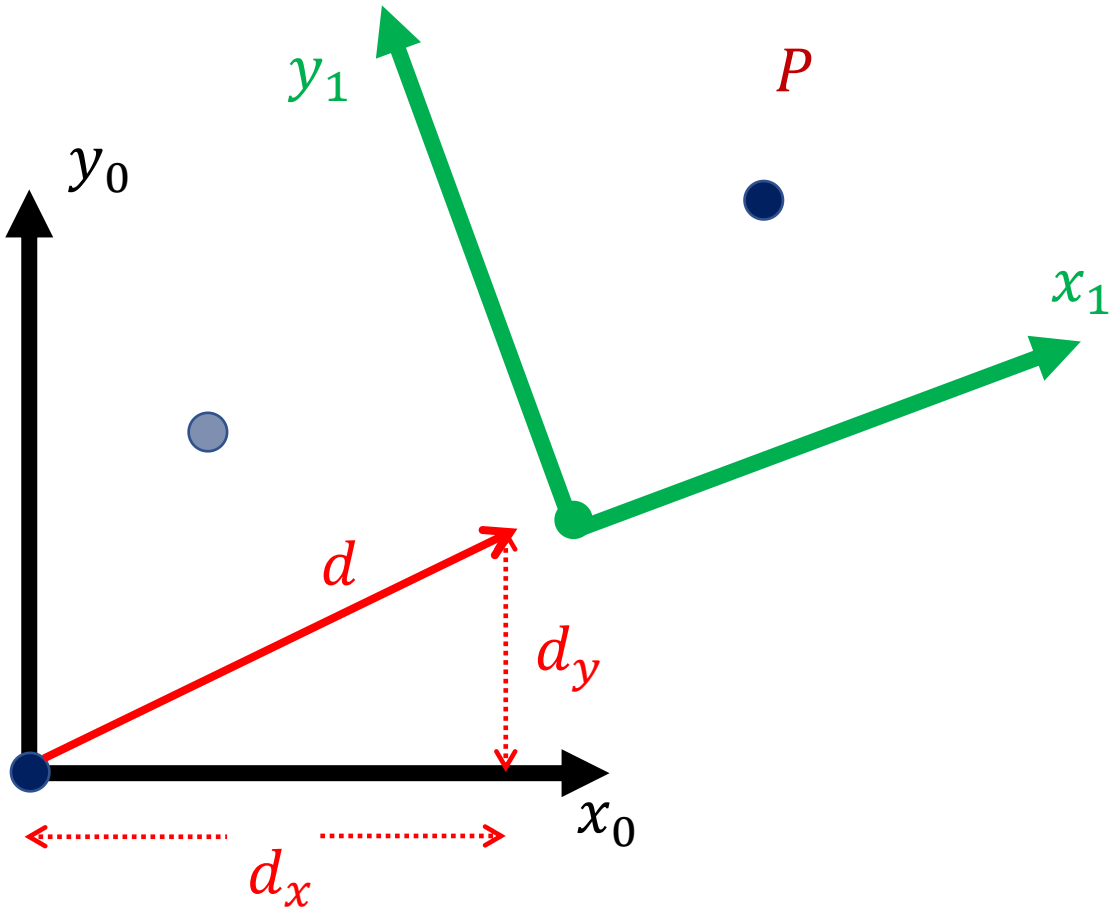
Kinematic Structure in 2D



$$T_{W\to\text{eff}}(q) = T_{W\to A}\, T_{A\to A'}(q)\, T_{A'\to B}\, T_{B\to B'}(q)\, T_{B'\to C}\, T_{C\to C'}(q)\, T_{C'\to\text{eff}}$$

2D Transformation (Recap)

Two reference Frame 0 and 1, a 2D point ^1P in reference frame 1

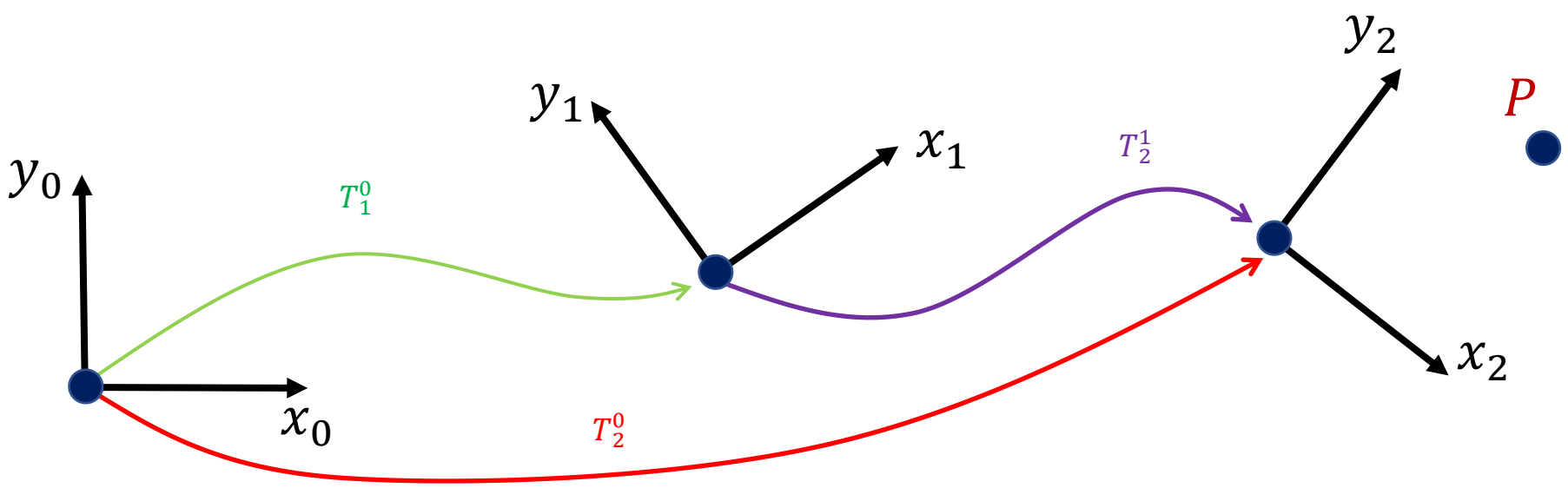


$$^0\mathbf{P} = {}^0\boldsymbol{T}_1\,{}^1\boldsymbol{P}$$

$$^0\boldsymbol{T}_1 = \begin{bmatrix} \boldsymbol{R}_1^0 & \boldsymbol{d}^0 \\ 0_2 & 1 \end{bmatrix}$$

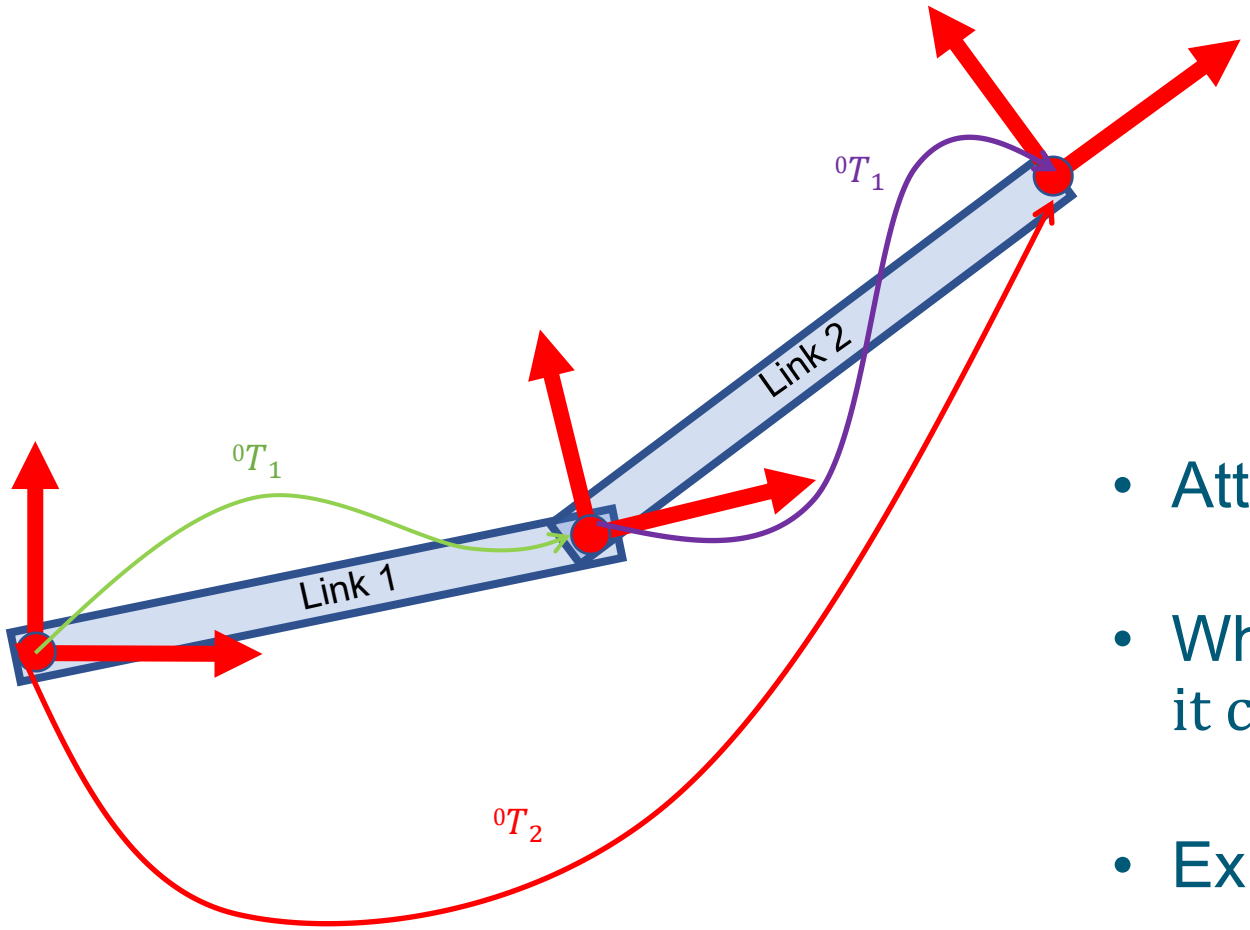
$$R = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \quad d = \begin{bmatrix} dx \\ dy \end{bmatrix}$$

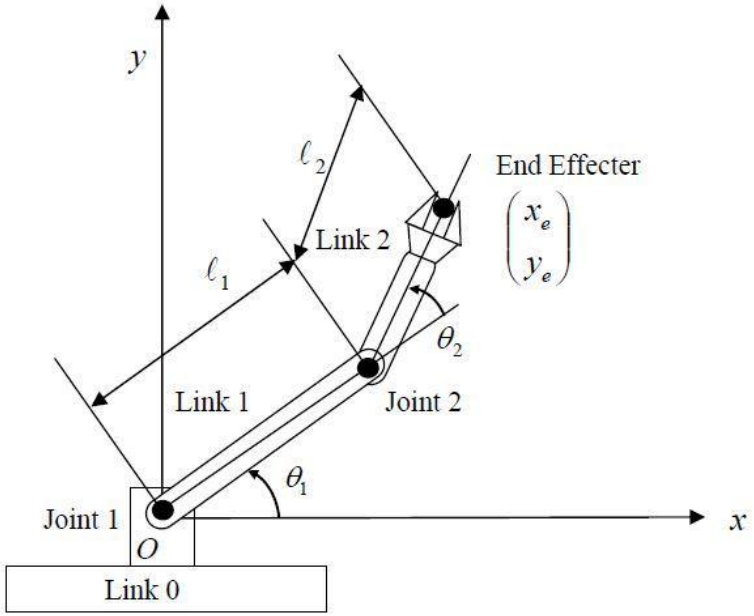
Composition of Transformations (Recap)



$$\left.\begin{aligned} \tilde{P}^0 &= {}^0\boldsymbol{T}_1 \tilde{P}^1 \\ \tilde{P}^1 &= {}^1\boldsymbol{T}_2 \tilde{P}^2 \end{aligned}\right\} \longrightarrow \tilde{P}^0 = {}^0\boldsymbol{T}_1 {}^1\boldsymbol{T}_2 \tilde{P}^2$$

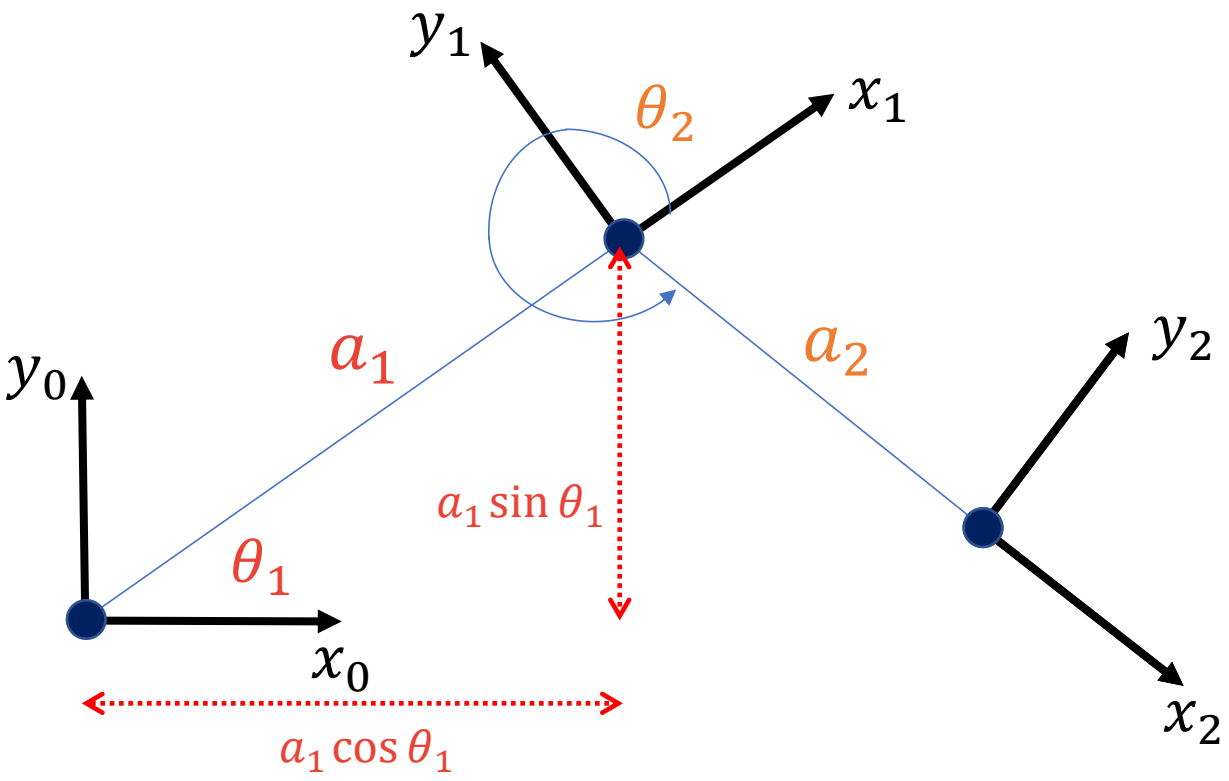
Two link Planar Arm

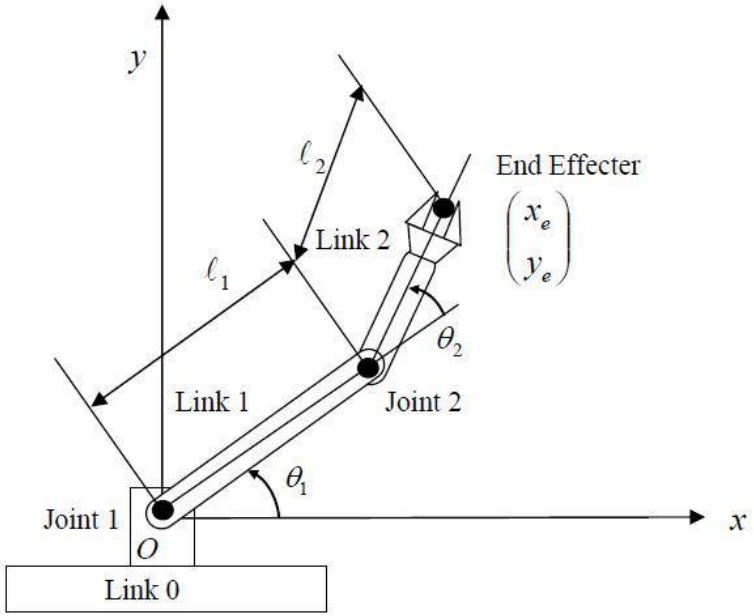




- Attach a coordinate frame to each link
- What we want to know is $^0\boldsymbol{T}_2$ and we know it can be computed by $^0\boldsymbol{T}_1{}^1\boldsymbol{T}_2$
- Express T_i^{i-1} as a function of θ_i

Two Link Case





Assume:

- a_i is the length of link i
- θ_i is rotation angle of joint i

$$ {}^{0}\boldsymbol{T}_1 = \begin{bmatrix} \cos\theta_1 & -\sin\theta_1 & a_1\cos\theta_1 \\ \sin\theta_1 & \cos\theta_1 & a_1\sin\theta_1 \\ 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{1}\boldsymbol{T}_2 = \begin{bmatrix} \cos\theta_2 & -\sin\theta_2 & a_2\cos\theta_2 \\ \sin\theta_2 & \cos\theta_2 & a_2\sin\theta_2 \\ 0 & 0 & 1 \end{bmatrix} $$

Forward Kinematic

- The forward kinematic map gives the position and orientation of the end-effector frame as a function of the joint angles

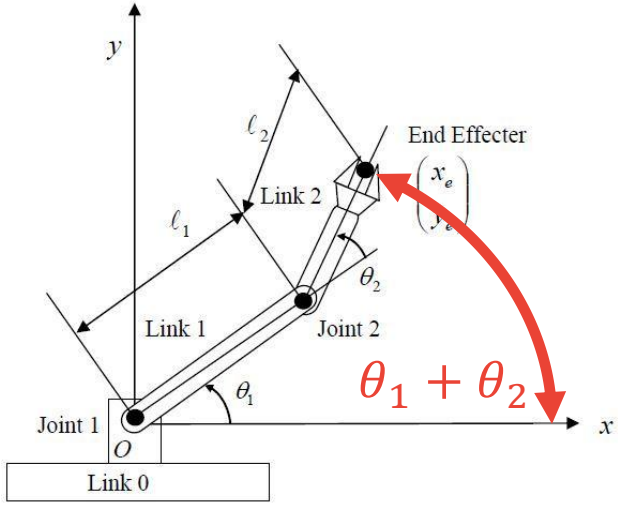
$$ {}^{0}T_n = F(q_1, \ldots, q_n) = {}^{0}T_1(q_1) \ldots {}^{n-1}T_n(q_n) $$

- For the two-link planar arm, we have

$$ {}^{0}T_2 = \begin{bmatrix} \cos\theta_1 & -\sin\theta_1 & a_1\cos\theta_1 \\ \sin\theta_1 & \cos\theta_1 & a_1\sin\theta_1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta_2 & -\sin\theta_2 & a_2\cos\theta_2 \\ \sin\theta_2 & \cos\theta_2 & a_2\sin\theta_2 \\ 0 & 0 & 1 \end{bmatrix} $$

$$ = \begin{bmatrix} \cos(\theta_1+\theta_2) & -\sin(\theta_1+\theta_2) & a_1\cos\theta_1 + a_2\cos(\theta_1+\theta_2) \\ \sin(\theta_1+\theta_2) & \cos(\theta_1+\theta_2) & a_1\sin\theta_1 + a_2\sin(\theta_1+\theta_2) \\ 0 & 0 & 1 \end{bmatrix} $$

Rotation with $\theta_1 + \theta_2$



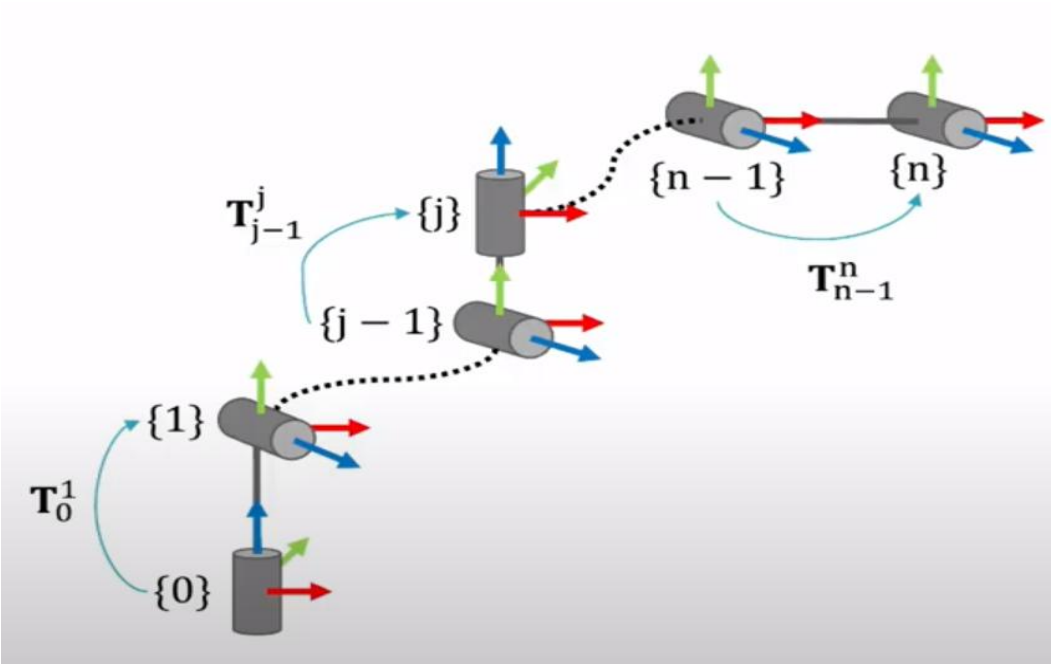
Question?

Forward Kinematic in 3D

- 6 parameters needed (x, y, z, roll, pitch, yaw) for each transformation matrix

$$\mathbf{T}_0^n = \mathbf{T}_0^1 \times \mathbf{T}_1^2 \times \mathbf{T}_2^3 \times \cdots \times \mathbf{T}_{n-1}^n$$

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_{1\times3} & 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & x \\ r_{21} & r_{22} & r_{23} & y \\ r_{31} & r_{32} & r_{33} & z \\ 0 & 0 & 0 & 1 \end{bmatrix} \in \mathbb{SE}(3)$$

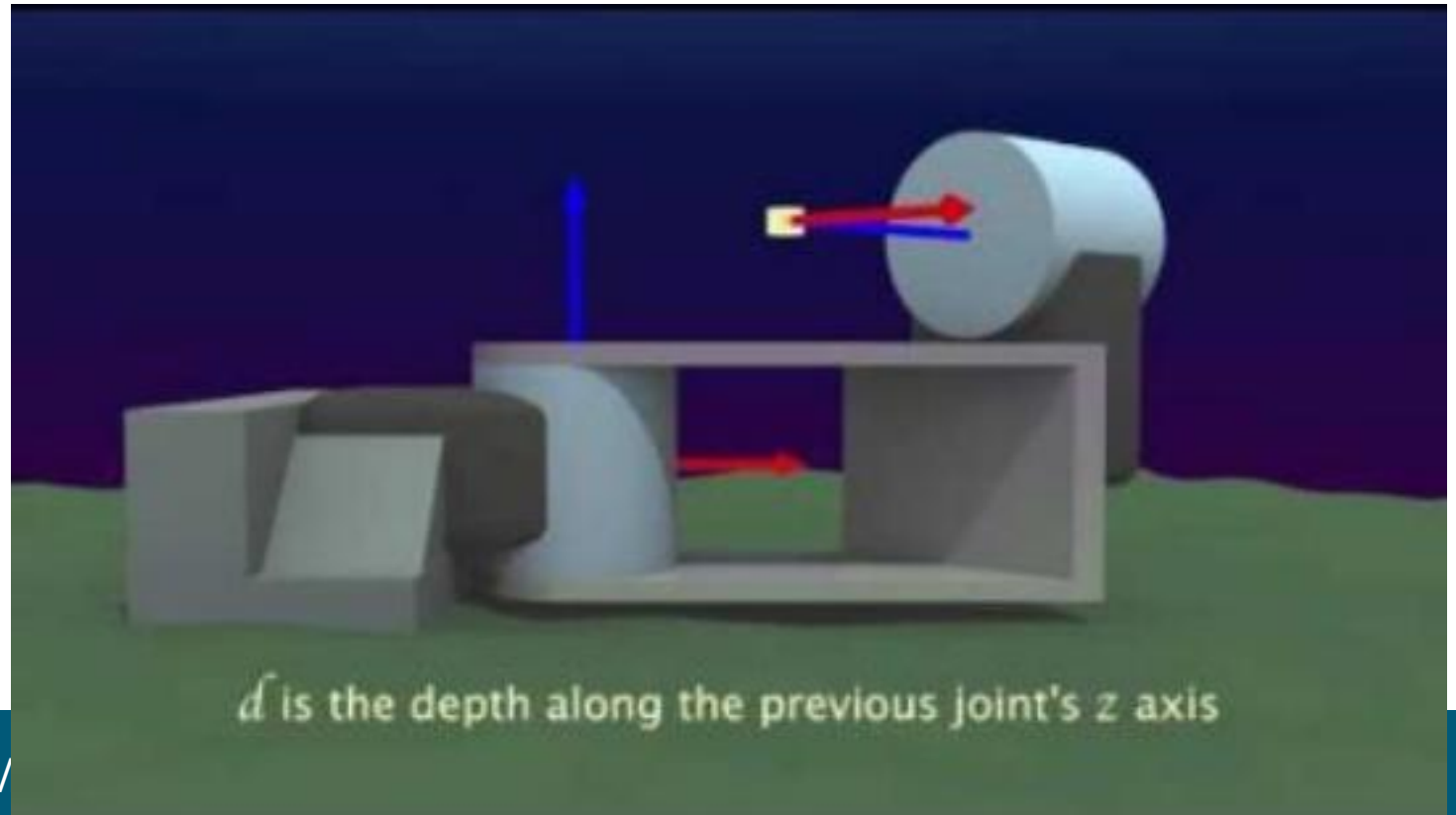


https://www.youtube.com/watch?v=09I15RO49vg

Can we represent the reference frame differently?

Forward Kinematics Using Denavit-Hartenberg (DH) Parameters

- Introduced by Jacques Denavit and Richard Hartenberg in 1955
- Minimum number of parameters to describe forward kinematics (**only 4 parameters needed)**



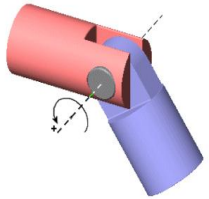
https://youtu.be/rA9tm0gTln8

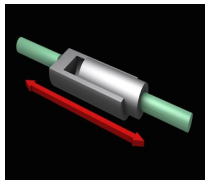
Forward Kinematics Using Denavit-Hartenberg (DH) Parameters

- Minimum number of parameters to describe forward kinematics (**only 4 parameters needed)**

One example: T^{i}_{i-1}

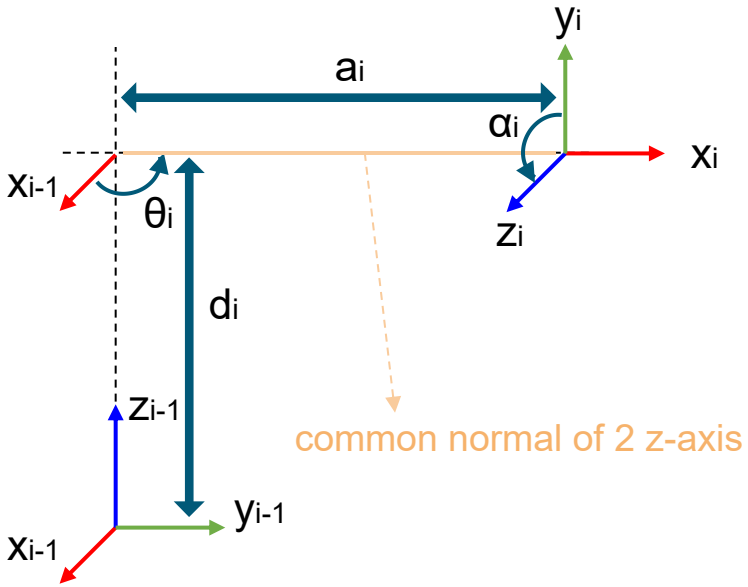
- θ_i : angle about z_{i-1}, make x_{i-1} align to x_i
- d_i : offset along z_{i-1} to the common normal
- a_i : length of the common normal
- α_i : angle about common normal (x_{i-1}, x_i), make z_{i-1} align to z_i





Note:

1. a_i and α_i are **fixed**
2. θ_i is variable if joint i is **revolute**
3. d_i is variable if joint i is **prismatic**

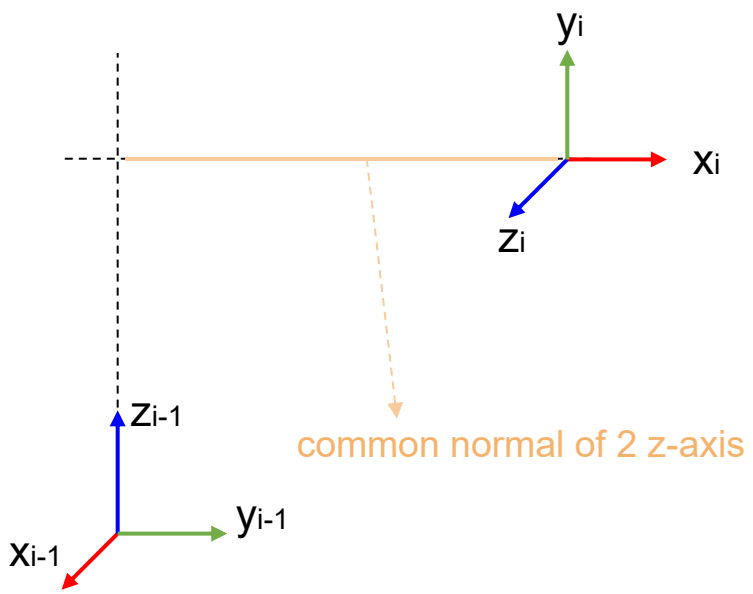


Forward Kinematics Using Denavit-Hartenberg (DH) Parameters

- Minimum number of parameters to describe forward kinematics (**only 4 parameters needed)**

One example: T^{i}_{i-1}

- θi : angle about zi-1, make xi-1 align to xi
- di : offset along zi-1 to the common normal
- ai : length of the common normal
- αi : angle about common normal (xi-1, xi), make zi-1 align to zi



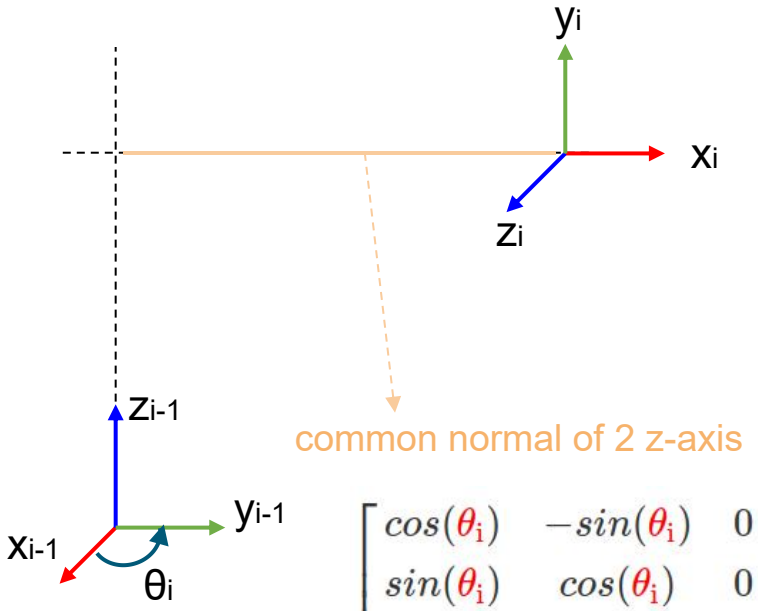
Forward Kinematics Using Denavit-Hartenberg (DH) Parameters

- Minimum number of parameters to describe forward kinematics (**only 4 parameters needed)**

One example: T^{i}_{i-1}

- θi : angle about zi-1, make xi-1 align to xi
- di : offset along zi-1 to the common normal
- ai : length of the common normal
- αi : angle about common normal (xi-1, xi), make zi-1 align to zi

$$T^{i}_{i-1} = T_{Rz}(\theta_i) \times$$



$$\begin{bmatrix} cos(\theta_i) & -sin(\theta_i) & 0 & 0 \\ sin(\theta_i) & cos(\theta_i) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

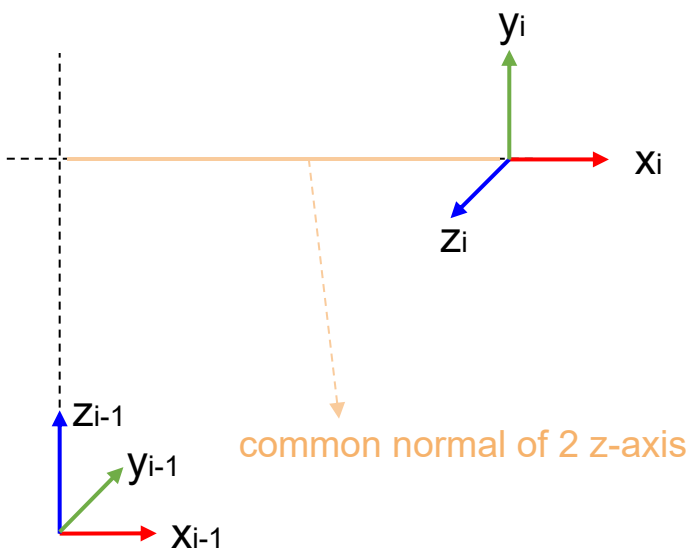
Forward Kinematics Using Denavit-Hartenberg (DH) Parameters

- Minimum number of parameters to describe forward kinematics (**only 4 parameters needed)**

One example: T^{i}_{i-1}

- θi : angle about zi-1, make xi-1 align to xi
- di : offset along zi-1 to the common normal
- ai : length of the common normal
- αi : angle about common normal (xi-1, xi), make zi-1 align to zi

$$T^{i}_{i-1} = T_{Rz}(\theta_i) \times$$



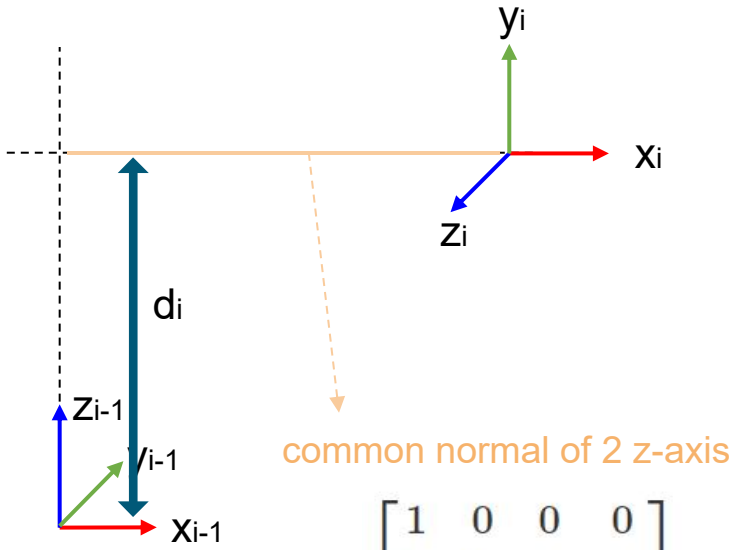
Forward Kinematics Using Denavit-Hartenberg (DH) Parameters

- Minimum number of parameters to describe forward kinematics (**only 4 parameters needed)**

One example: T^{i}_{i-1}

- θi : angle about zi-1, make xi-1 align to xi
- di : offset along zi-1 to the common normal
- ai : length of the common normal
- αi : angle about common normal (xi-1, xi), make zi-1 align to zi

$$T^{i}_{i-1} = T_{Rz}(\theta_i) \times T_z(d_i) \times$$



$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

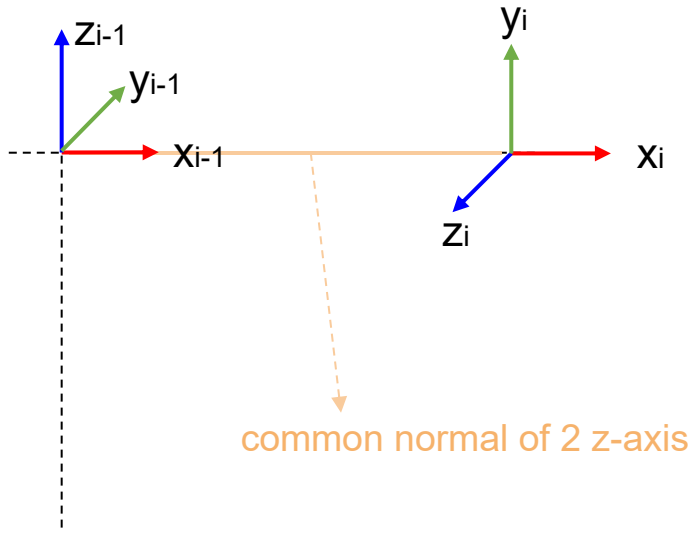
Forward Kinematics Using Denavit-Hartenberg (DH) Parameters

- Minimum number of parameters to describe forward kinematics (**only 4 parameters needed)**

One example: T^{i}_{i-1}

- θi : angle about zi-1, make xi-1 align to xi
- di : offset along zi-1 to the common normal
- ai : length of the common normal
- αi : angle about common normal (xi-1, xi), make zi-1 align to zi

$$T^{i}_{i-1} = T_{Rz}(\theta_i) \times T_z(d_i) \times$$

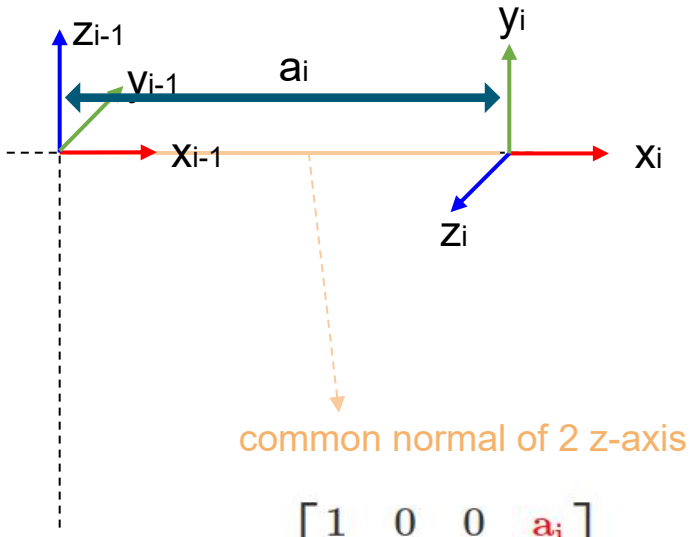


Forward Kinematics Using Denavit-Hartenberg (DH) Parameters

- Introduced by Jacques Denavit and Richard Hartenberg in 1955
- Minimum number of parameters to describe forward kinematics (**only 4 parameters needed)**

One example: T^{i}_{i-1}

- θi : angle about zi-1, make xi-1 align to xi
- di : offset along zi-1 to the common normal
- ai : length of the common normal
- αi : angle about common normal (xi-1, xi), make zi-1 align to zi



$$T^{i}_{i-1} = T_{Rz}(\theta_i) \times T_z(d_i) \times T_x(a_i) \times$$

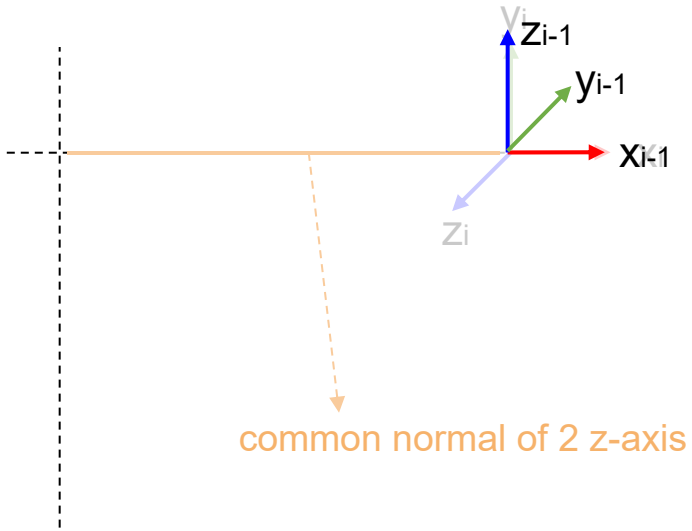
$$\begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Forward Kinematics Using Denavit-Hartenberg (DH) Parameters

- Introduced by Jacques Denavit and Richard Hartenberg in 1955
- Minimum number of parameters to describe forward kinematics (**only 4 parameters needed)**

One example: T^{i}_{i-1}

- θi : angle about zi-1, make xi-1 align to xi
- di : offset along zi-1 to the common normal
- ai : length of the common normal
- αi : angle about common normal (xi-1, xi), make zi-1 align to zi



$$T^{i}_{i-1} = T_{Rz}(\theta_i) \times T_z(d_i) \times T_x(a_i) \times$$

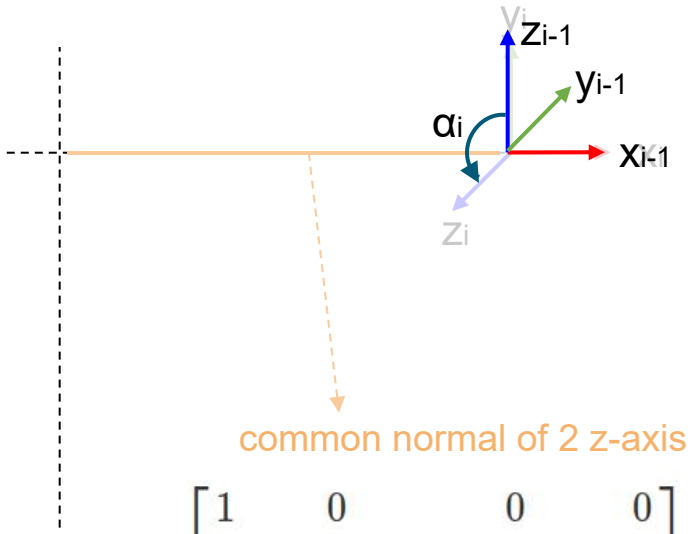
Forward Kinematics Using Denavit-Hartenberg (DH) Parameters

- Introduced by Jacques Denavit and Richard Hartenberg in 1955
- Minimum number of parameters to describe forward kinematics (**only 4 parameters needed)**

One example: T^{i}_{i-1}

- θi : angle about zi-1, make xi-1 align to xi
- di : offset along zi-1 to the common normal
- ai : length of the common normal
- αi : angle about common normal (xi-1, xi), make zi-1 align to zi

$$T^{i}_{i-1} = T_{Rz}(\theta_i) \times T_z(d_i) \times T_x(a_i) \times T_{Rx}(\alpha_i)$$



$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos(\alpha_i) & -sin(\alpha_i) & 0 \\ 0 & sin(\alpha_i) & cos(\alpha_i) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

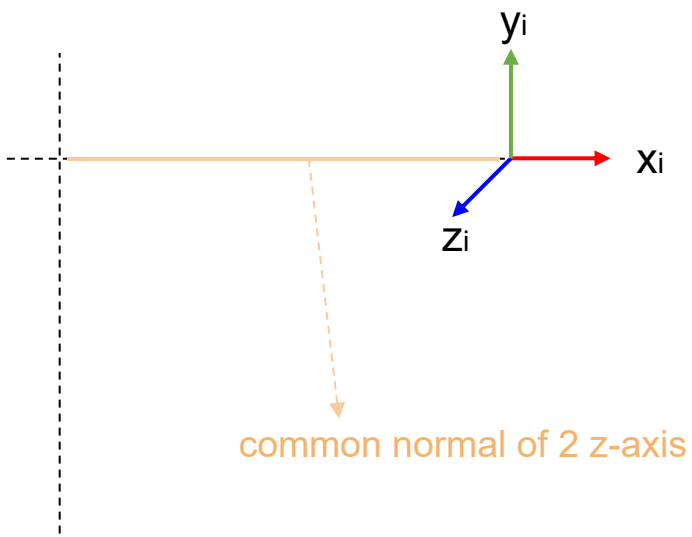
Forward Kinematics Using Denavit-Hartenberg (DH) Parameters

- Introduced by Jacques Denavit and Richard Hartenberg in 1955
- Minimum number of parameters to describe forward kinematics (**only 4 parameters needed)**

One example: T^{i}_{i-1}

- θi : angle about zi-1, make xi-1 align to xi
- di : offset along zi-1 to the common normal
- ai : length of the common normal
- αi : angle about common normal (xi-1, xi), make zi-1 align to zi

$$T^{i}_{i-1} = T_{Rz}(\theta_i) \times T_z(d_i) \times T_x(a_i) \times T_{Rx}(\alpha_i)$$



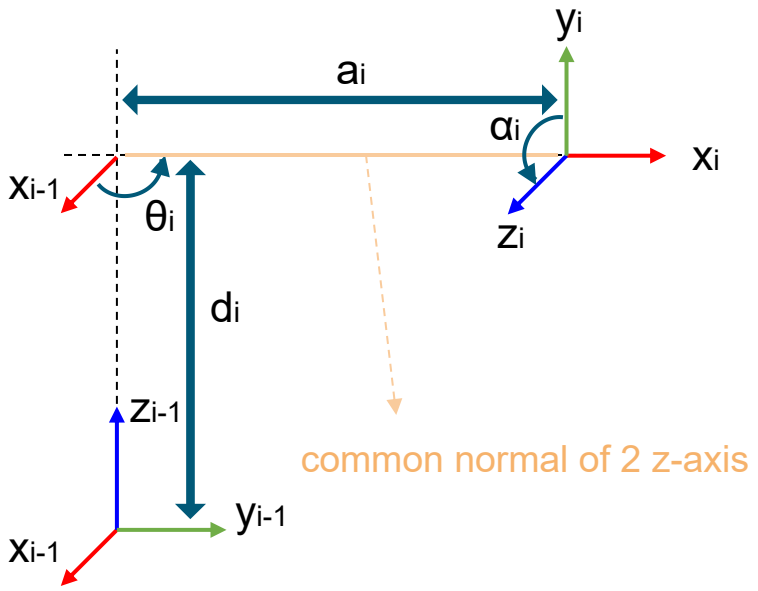
Forward Kinematics Using Denavit-Hartenberg (DH) Parameters

Transformation matrix expansion

$$T^{i}_{i-1} = T_{Rz}(\theta_i) \times T_z(d_i) \times T_x(a_i) \times T_{Rx}(\alpha_i)$$

$$= \begin{bmatrix} cos(\theta_i) & -sin(\theta_i) & 0 & 0 \\ sin(\theta_i) & cos(\theta_i) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos(\alpha_i) & -sin(\alpha_i) & 0 \\ 0 & sin(\alpha_i) & cos(\alpha_i) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} cos(\theta_i) & -sin(\theta_i)cos(\alpha_i) & sin(\theta_i)sin(\alpha_i) & a_i cos(\theta_i) \\ sin(\theta_i) & cos(\theta_i)cos(\alpha_i) & -cos(\theta_i)sin(\alpha_i) & a_i sin(\theta_i) \\ 0 & sin(\alpha_i) & cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

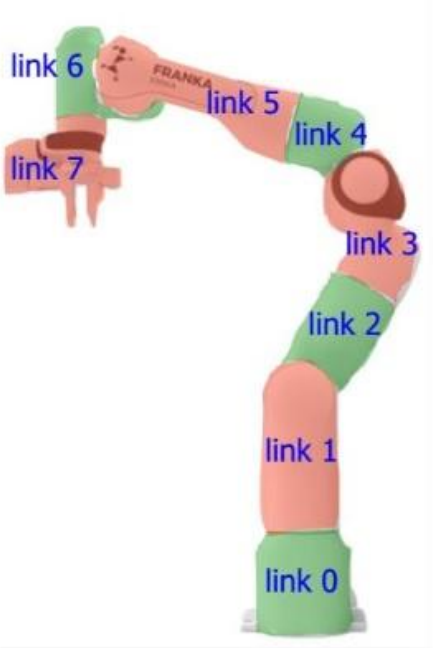


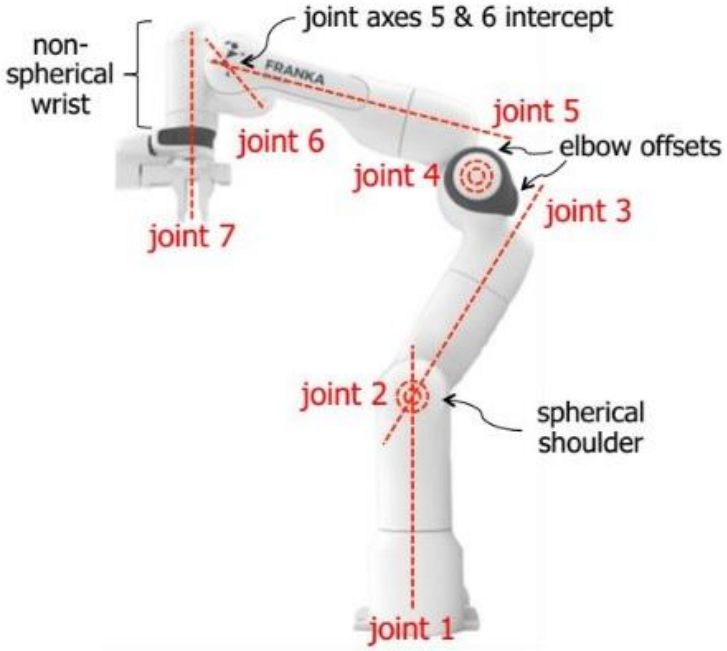
Then for the end-effecgtor frame n:

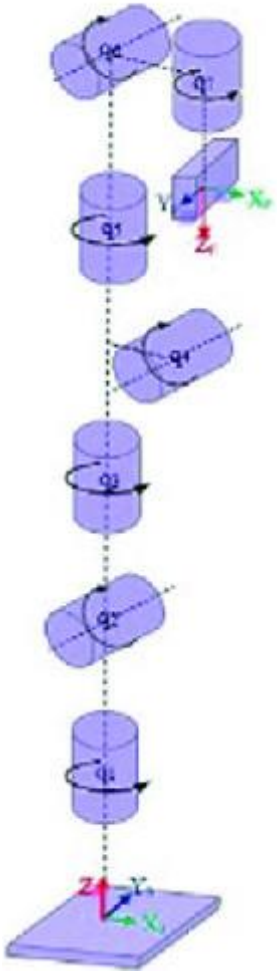
$$T^{n}_{0} = \prod_{i=1}^{n} T^{i}_{i-1}$$

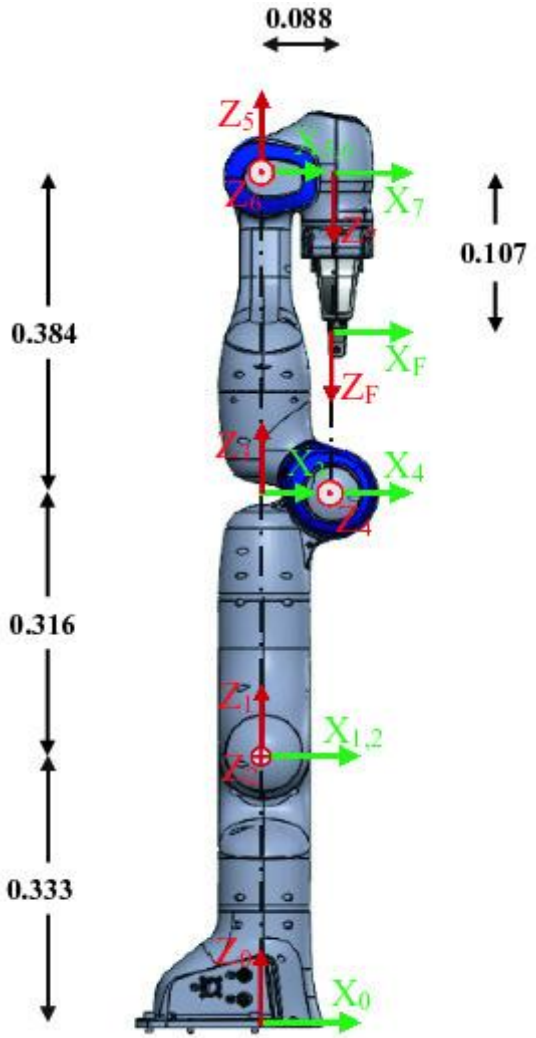
Franka Emika

Use modified D-H conventions







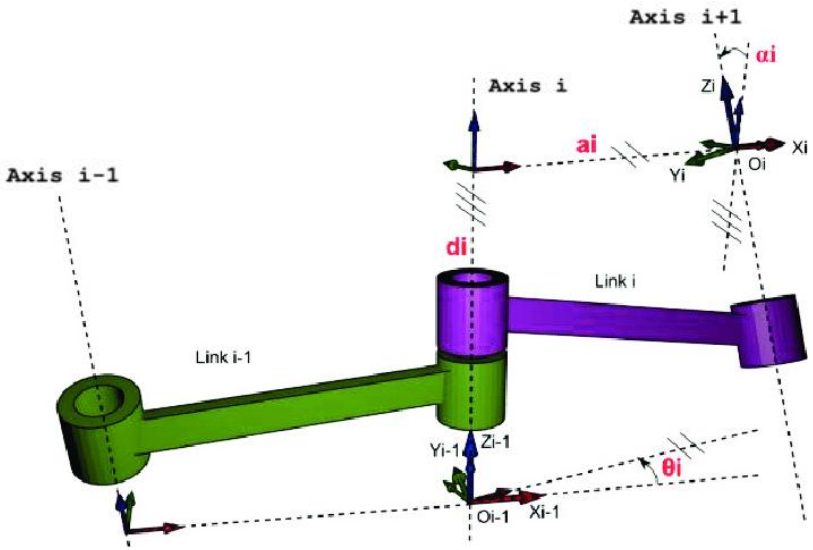


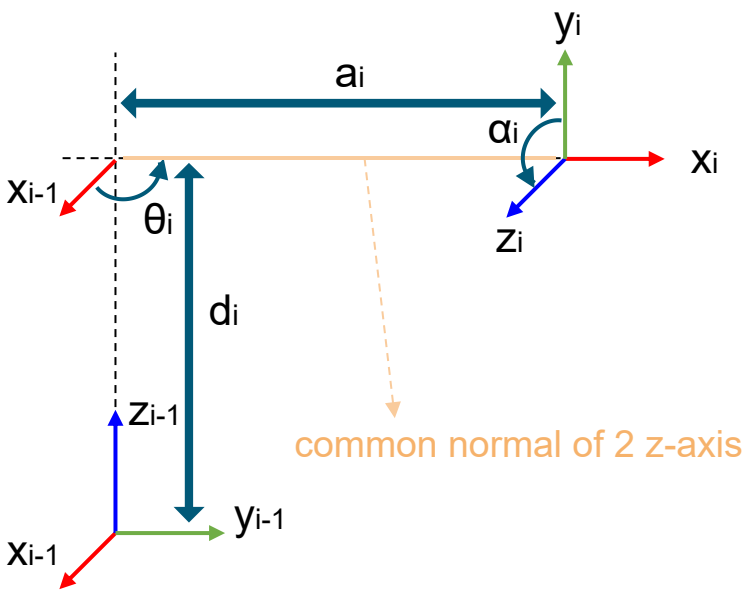
shorturl.at/cjv06

https://link.springer.com/chapter/10.1007/978-3-030-75259-0_11

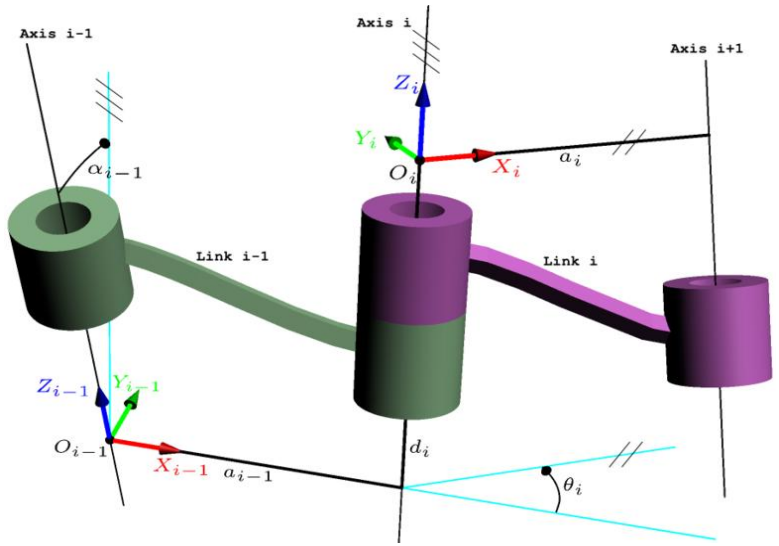
Two conventions for the D-H parameters

Classic D-H Convention





Modified D-H Convention



Notes

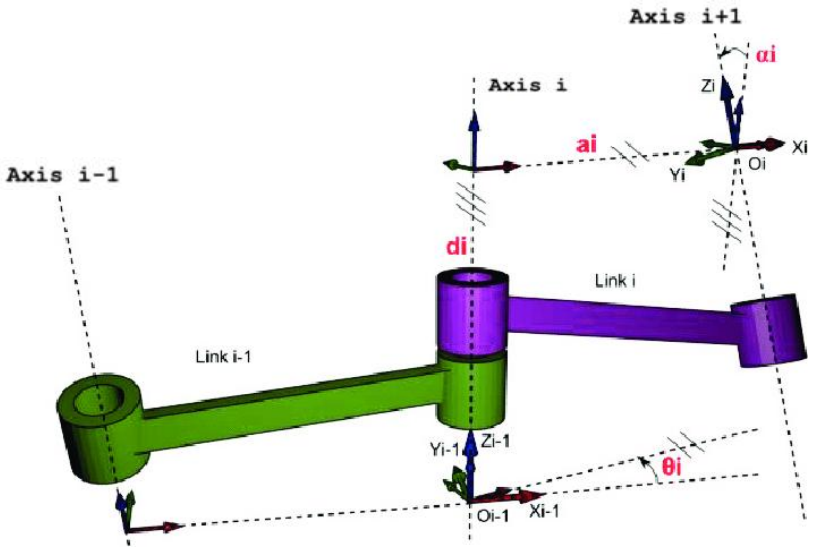
- coordinates of **Oi** is put **on axis i+1** in classic D-H convention
- this is what we have learned in previous slides

Notes

- coordinates of **Oi** is put **on the axis i**, **not the axis i+1** in modified D-H convention

Two conventions and their constraints

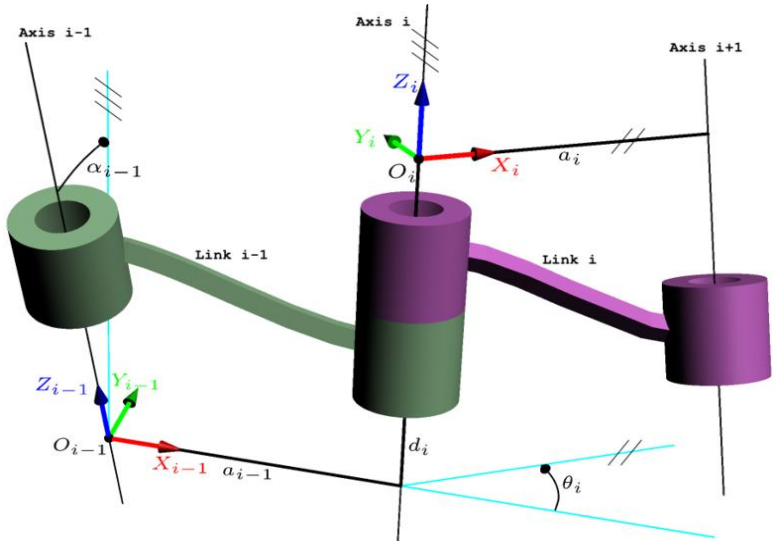
Classic D-H Convention



Constraints

- the **X_i-axis** is perpendicular to the **Z_{i-1}**
- the **X_i-axis** intersects **Z_{i-1}**
- the origin of joint **i** is at the intersection of **X_i** and **Z_i**

Modified D-H Convention



Constraints

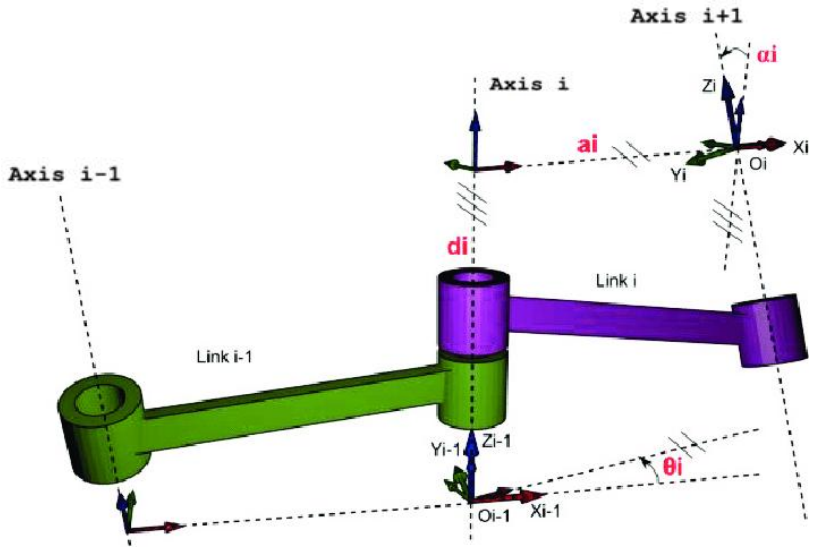
- the **Z_i-axis** is perpendicular to the **X_{i-1}**
- the **Z_i-axis** intersects **X_{i-1}**
- the origin of joint **i** is at the intersection of **X_i** and **Z_i**

Classic D-H Conventions

$$T^{i}_{i\text{-}1} = T_{Rz}(\theta_i) \times T_z(d_i) \times T_x(a_i) \times T_{Rx}(\alpha_i)$$

$$= \begin{bmatrix} cos(\theta_i) & -sin(\theta_i) & 0 & 0 \\ sin(\theta_i) & cos(\theta_i) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos(\alpha_i) & -sin(\alpha_i) & 0 \\ 0 & sin(\alpha_i) & cos(\alpha_i) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} cos(\theta_i) & -sin(\theta_i)cos(\alpha_i) & sin(\theta_i)sin(\alpha_i) & a_i cos(\theta_i) \\ sin(\theta_i) & cos(\theta_i)cos(\alpha_i) & -cos(\theta_i)sin(\alpha_i) & a_i sin(\theta_i) \\ 0 & sin(\alpha_i) & cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



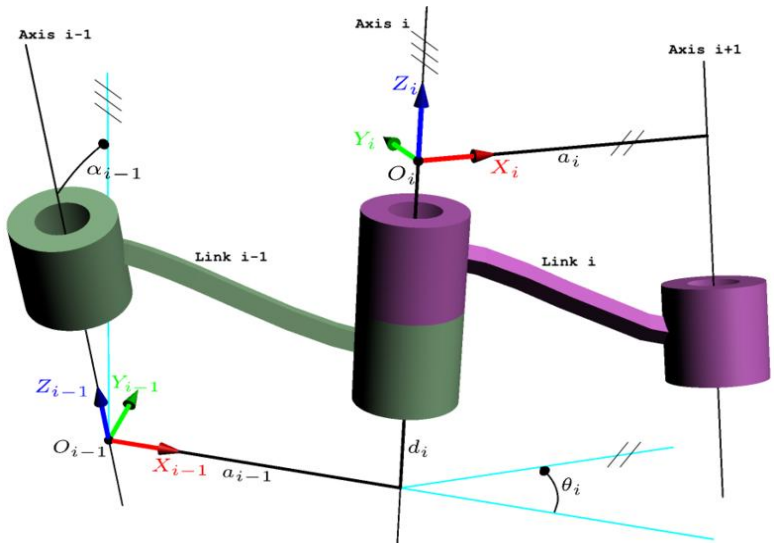
Then for the end-effecgtor frame n: $T^{n}_{0} = \prod_{i=1}^{n} T^{i}_{i-1}$

Modified D-H Conventions

$$T^{i}_{i-1} = T_{Rx}(\alpha_i) \times T_x(a_i) \times T_{Rz}(\theta_i) \times T_z(d_i)$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos(\alpha_i) & -sin(\alpha_i) & 0 \\ 0 & sin(\alpha_i) & cos(\alpha_i) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} cos(\theta_i) & -sin(\theta_i) & 0 & 0 \\ sin(\theta_i) & cos(\theta_i) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

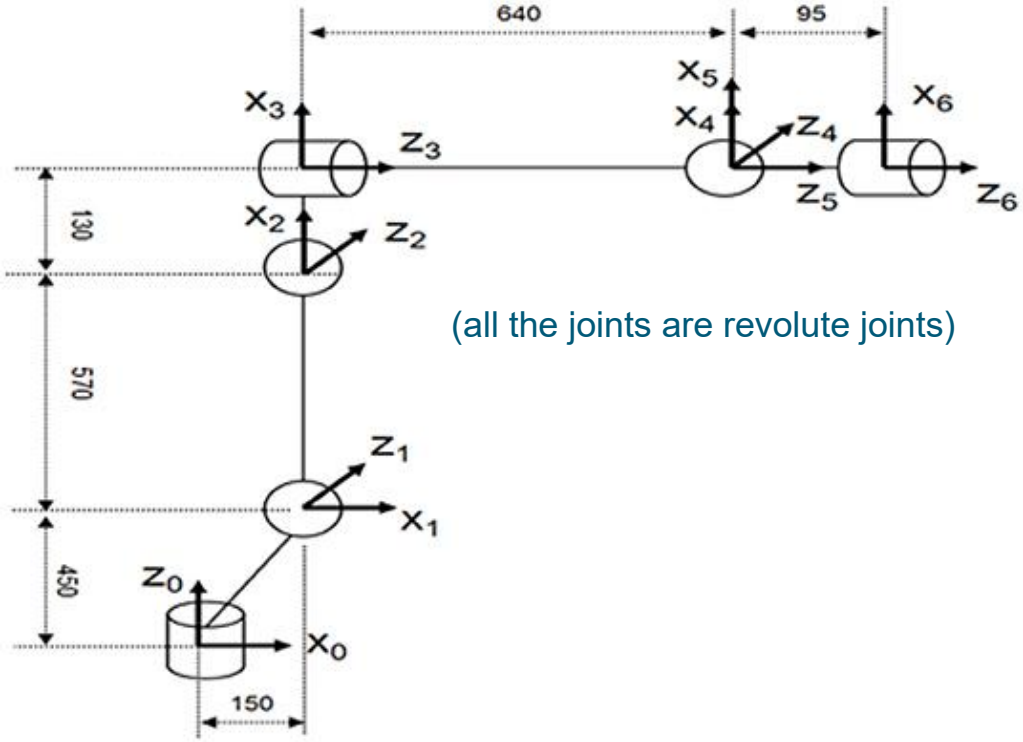
$$= \begin{bmatrix} cos(\theta_i) & -sin(\theta_i) & 0 & a_i \\ sin(\theta_i)cos(\alpha_i) & cos(\theta_i)cos(\alpha_i) & -sin(\alpha_i) & -d_i sin(\alpha_i) \\ sin(\theta_i)sin(\alpha_i) & cos(\theta_i)sin(\alpha_i) & cos(\alpha_i) & d_i cos(\alpha_i) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Then for the end-effecgtor frame n: $T^{n}_{0} = \prod_{i=1}^{n} T^{i}_{i-1}$

Quiz Time

- Fill in the DH table using **classic D-H convention**



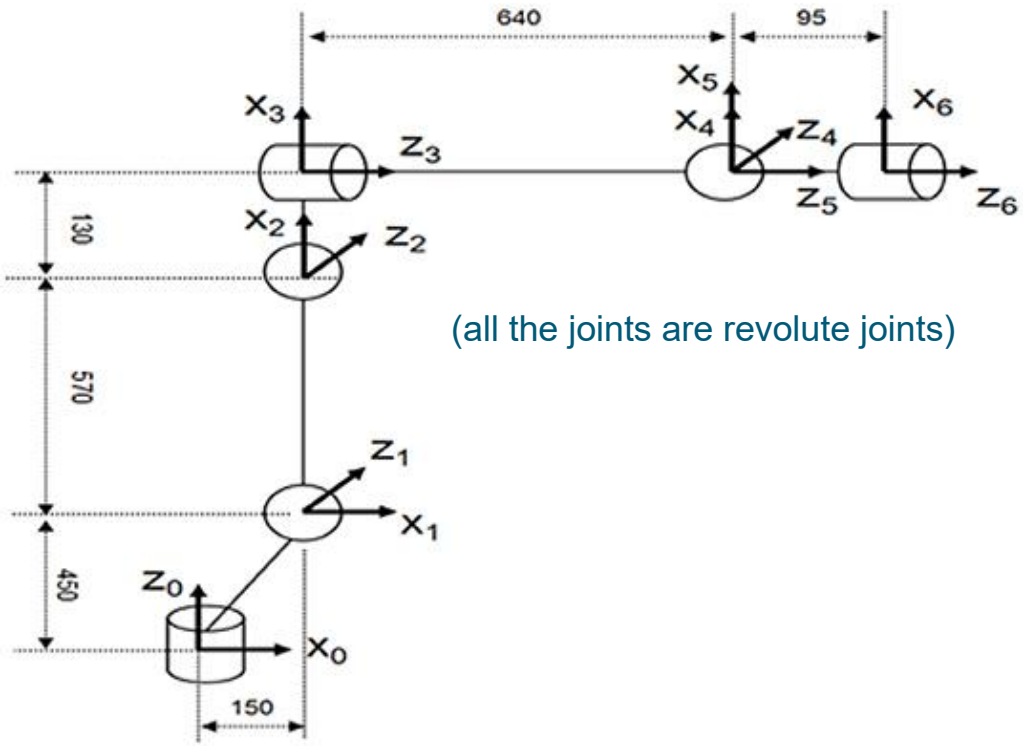
(all the joints are revolute joints)

joint variable

i	$d_i(mm)$	$\alpha_i(Deg)$	$a_i(mm)$	$\theta_i(Deg)$
1	450	−90	150	θ_1
2				
3				
4				
5				
6				

Quiz Time

- Fill in the DH table using **classic D-H convention**



(all the joints are revolute joints)

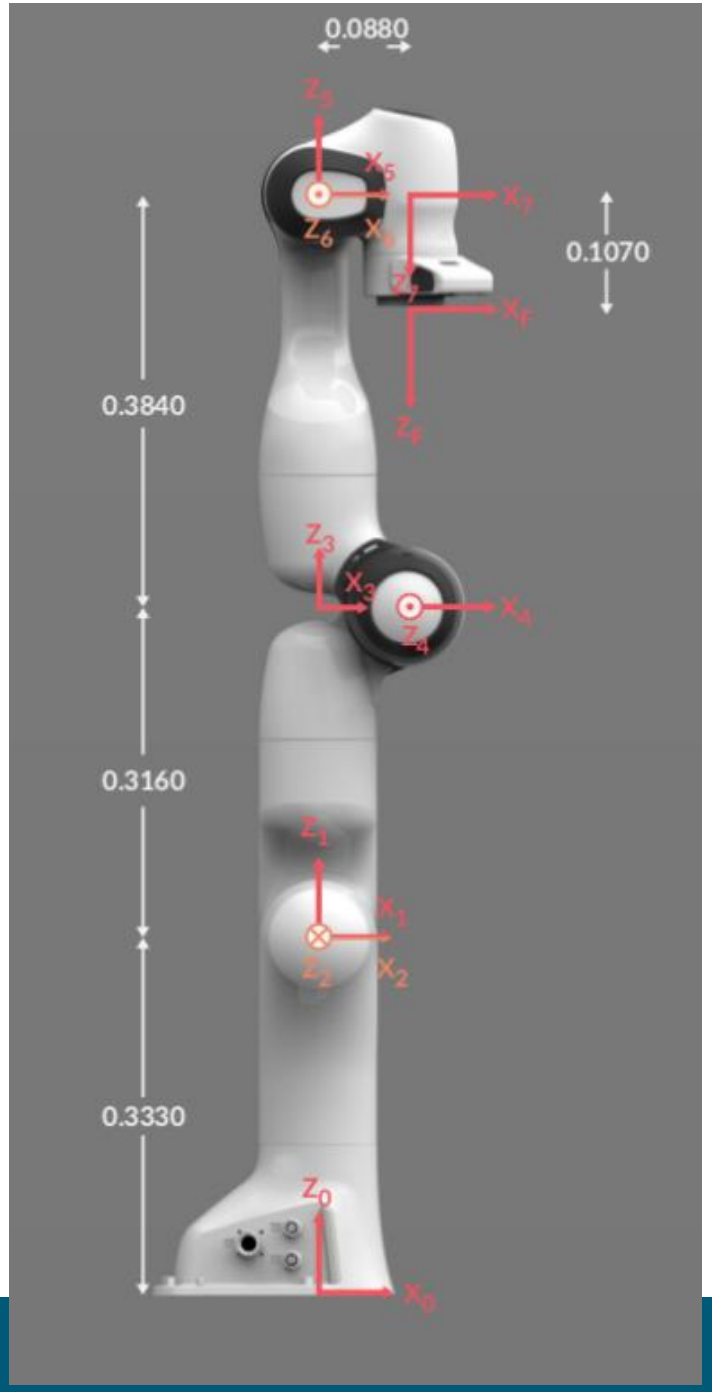
i	$d_i(mm)$	$\alpha_i(Deg)$	$a_i(mm)$	$\theta_i(Deg)$
1	450	−90	150	θ_1
2	0	0	570	θ_2
3	0	−90	130	θ_3
4	640	90	0	θ_4
5	0	−90	0	θ_5
6	95	0	0	θ_6

Quiz Time

- Fill up the DH table using **Modified DH convention**

Joint	a (m)	d (m)	α (rad)	θ (rad)
Joint 1	0	0.333	0	θ_1
Joint 2				
Joint 3				
Joint 4				
Joint 5				
Joint 6				
Joint 7				
Flange				

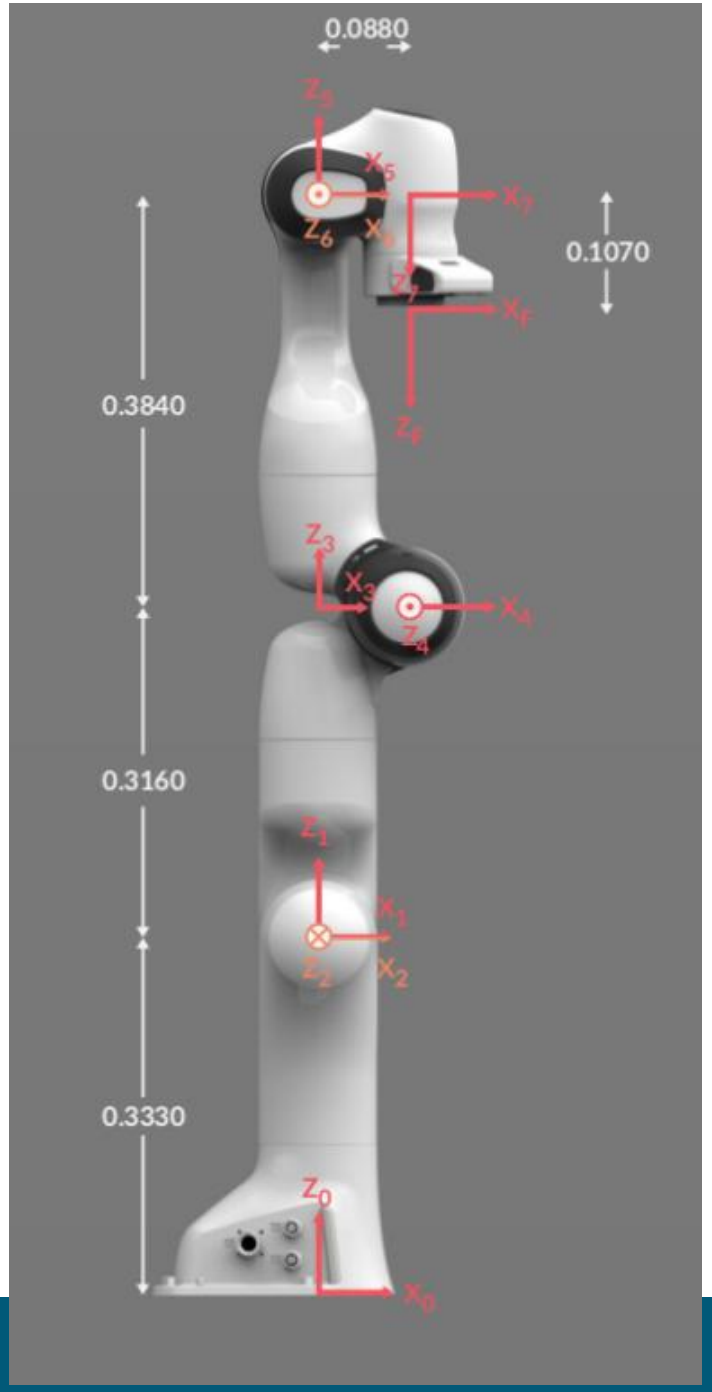
(all the joints are revolute joints)



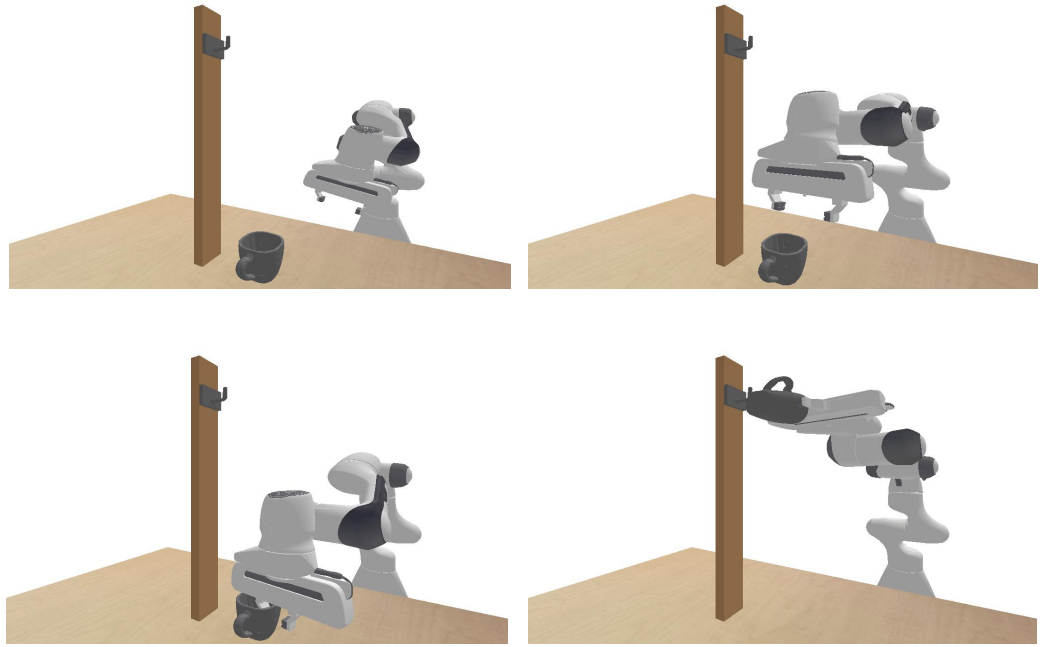
Quiz Time

- Fill up the DH table using **Modified DH convention**

Joint	a (m)	d (m)	α (rad)	θ (rad)
Joint 1	0	0.333	0	θ_1
Joint 2	0	0	$-\frac{\pi}{2}$	θ_2
Joint 3	0	0.316	$\frac{\pi}{2}$	θ_3
Joint 4	0.0825	0	$\frac{\pi}{2}$	θ_4
Joint 5	-0.0825	0.384	$-\frac{\pi}{2}$	θ_5
Joint 6	0	0	$\frac{\pi}{2}$	θ_6
Joint 7	0.088	0	$\frac{\pi}{2}$	θ_7
Flange	0	0.107	0	0



The Concept Map for Robot Manipulation



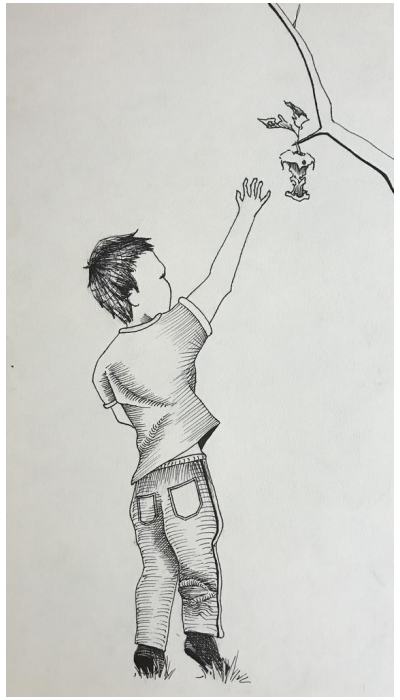
- Forward Kinematics
- 6D Pose Estimation
- Inverse Kinematics
- Grasping

Inverse Kinematic

Given a desired end-effector pose x^d

Solve for $q_1, \ldots, q_n$ to achieve a desired T^d i.e., find $q_1^d, \ldots, q_n^d$ such that $F(q_1^d, \ldots, q_n^d) = x^d$

We (human) are computing IK almost all the time (whenever we try to reach for anything). We can do this IK computation extremely efficient and robust



A Challenging Problem!

Inverse Kinematic

- For the two-link arm, the goal is to place the end-effector at a desired location $(o_{2,x}, o_{2,y})$
- Task: Solve for θ_1 and θ_2 such that:

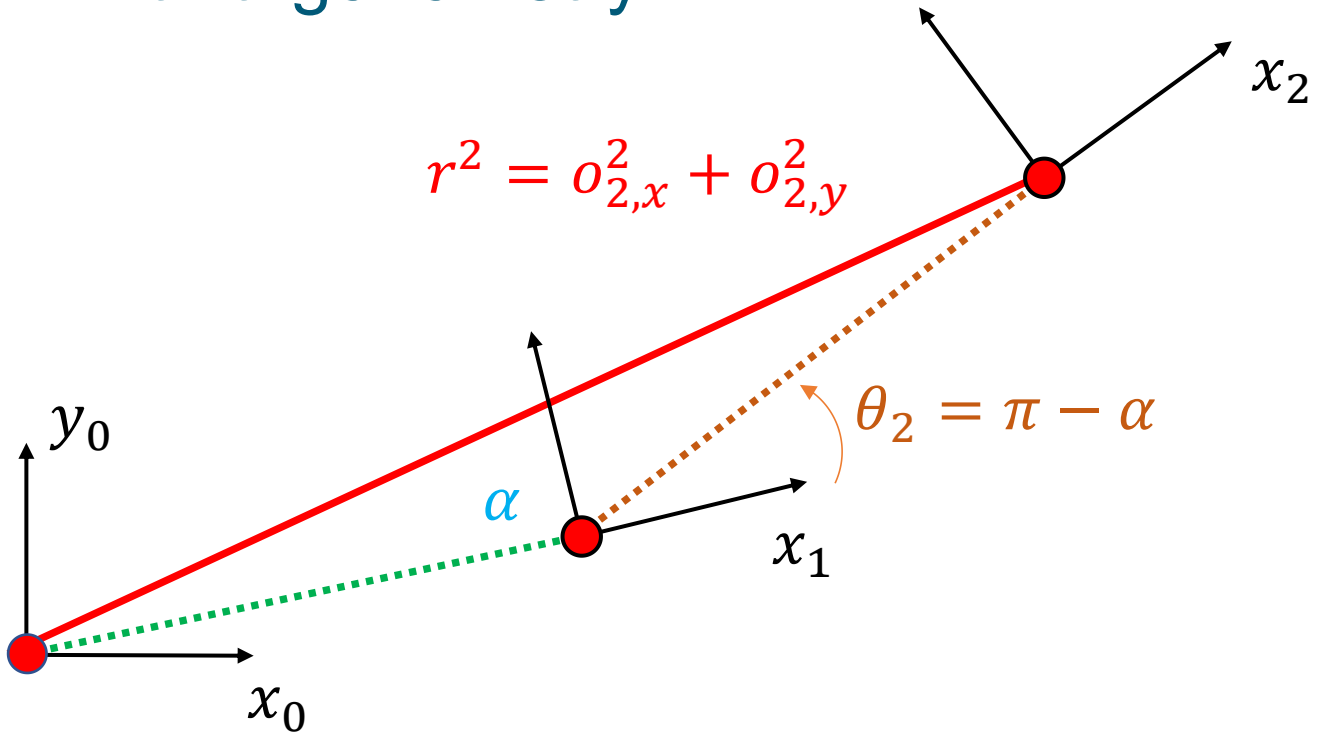
$$o_{2,x} = a_1 \cos\theta_1 + a_2 \cos(\theta_1 + \theta_2)$$
$$o_{2,y} = a_1 \sin\theta_1 + a_2 \sin(\theta_1 + \theta_2)$$

$${}^0T_2 = \begin{bmatrix} \cos(\theta_1+\theta_2) & -\sin(\theta_1+\theta_2) & a_1\cos\theta_1 + a_2\cos(\theta_1+\theta_2) \\ \sin(\theta_1+\theta_2) & \cos(\theta_1+\theta_2) & a_1\sin\theta_1 + a_2\sin(\theta_1+\theta_2) \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} * & * & o_{2,x} \\ * & * & o_{2,y} \\ 0 & 0 & 1 \end{bmatrix}$$

- Recall that a_1 and a_2 are constants defined by the mechanical structure of the arm
- This is a nonlinear set of equations in θ_1 and θ_2 --- and nonlinear equations can be very difficult to solve!

Geometric Methods

- IK for any two adjacent links that are **coplanar** can be solved with trigonometry



Denote the coordinates of the origin of frame 2 by $o_{2,x}, o_{2,y}$

The Law of Cosines:

$$r^2 = a_1^2 + a_2^2 - 2a_1a_2\cos\alpha$$

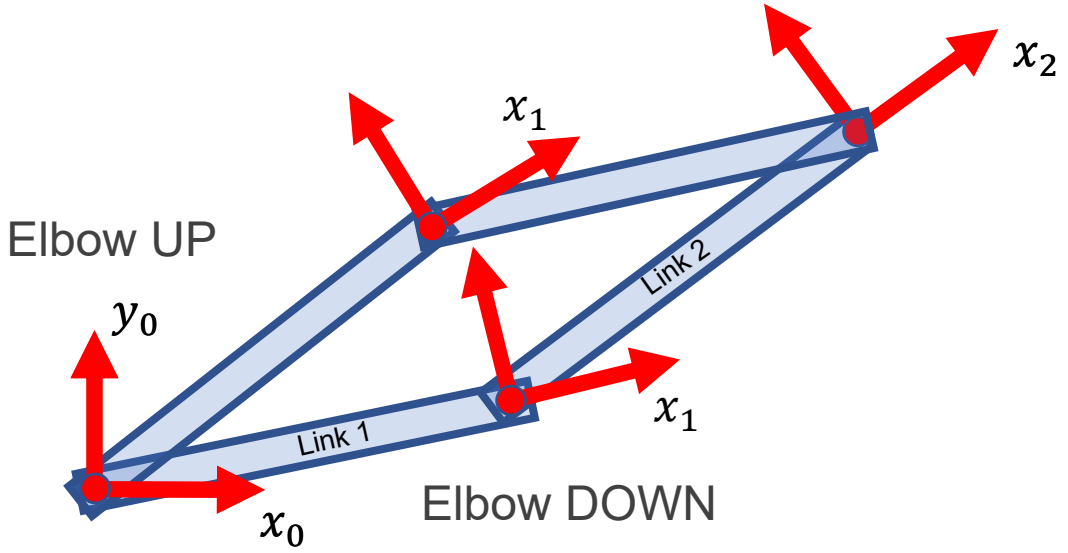
Define: $D \stackrel{\text{def}}{=} \frac{a_1^2+a_2^2-r^2}{2a_1a_2} = \cos\alpha$

Then $\sin\alpha = \pm\sqrt{1-D^2}$

Finally, $\alpha = \tan^{-1}\frac{\pm\sqrt{1-D^2}}{D}$

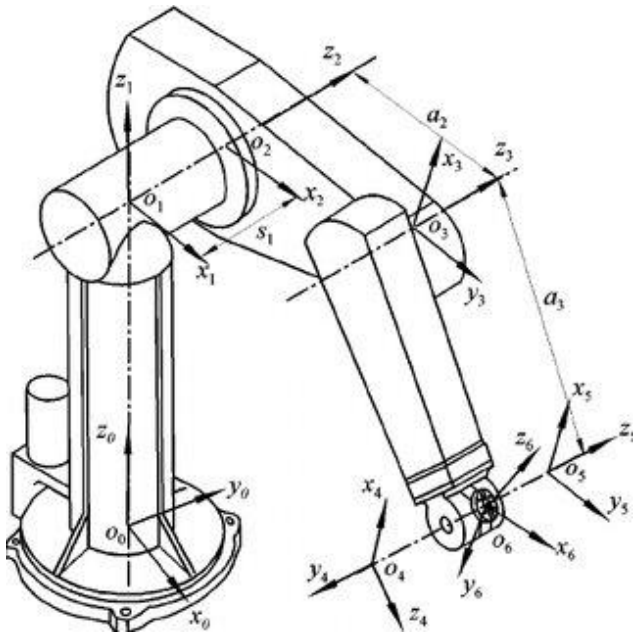
Uniqueness

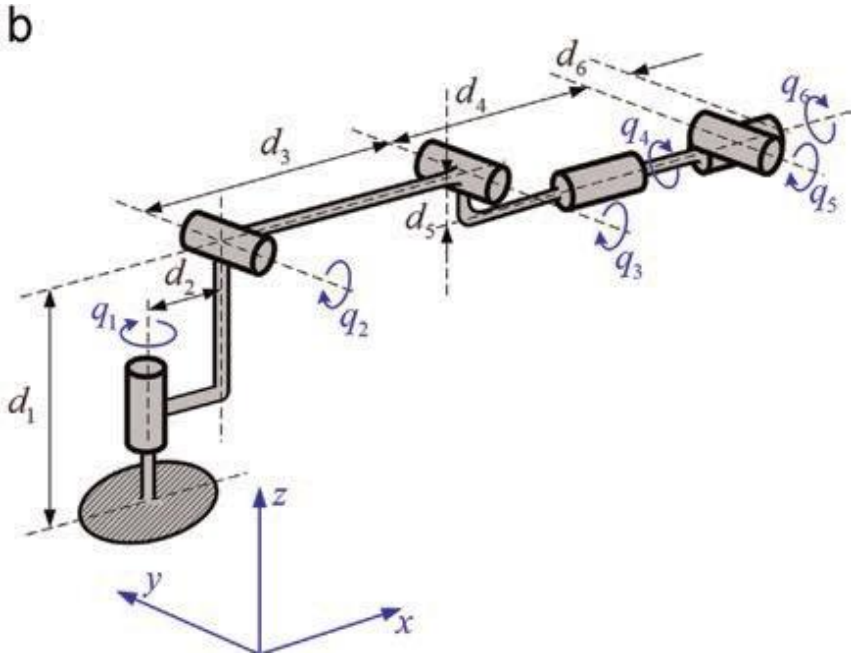
- Is the solution unique for $\alpha = \tan^{-1} \frac{\pm\sqrt{1-D^2}}{D}$?
  - No, we may choose either square root
- The second solution uses $\alpha = tan^{-1} \frac{-\sqrt{1-D^2}}{D}$ which results in an "elbow UP" configuration



More General Robot Arms

- No closed form solution

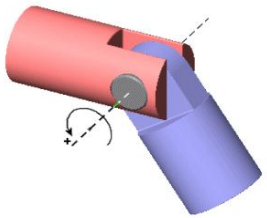


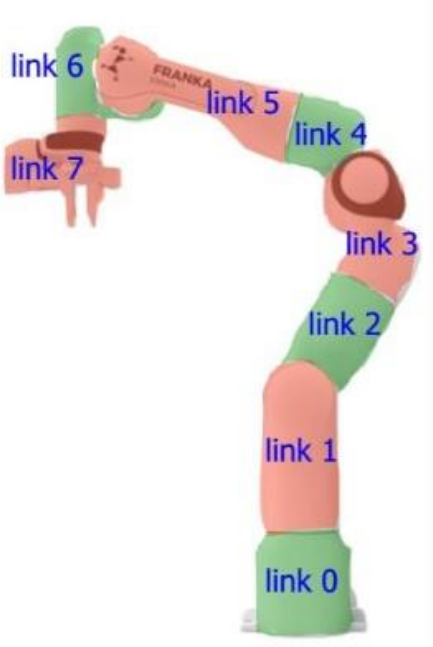


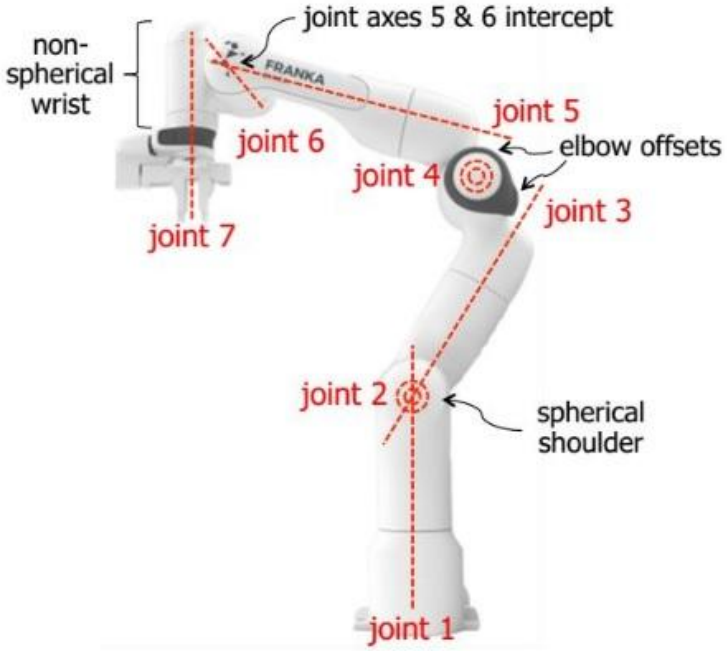
Iterative Methods for Inverse Kinematics

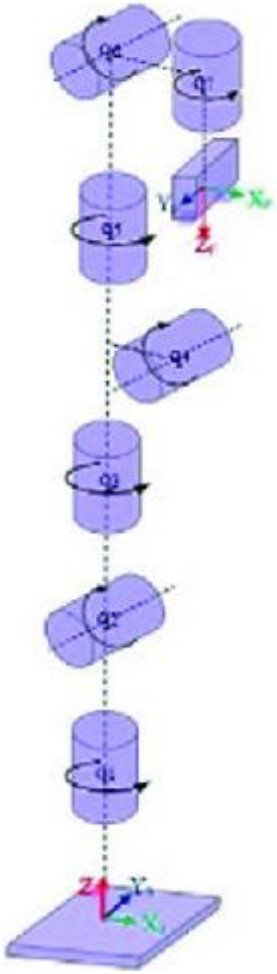
Franka Emika

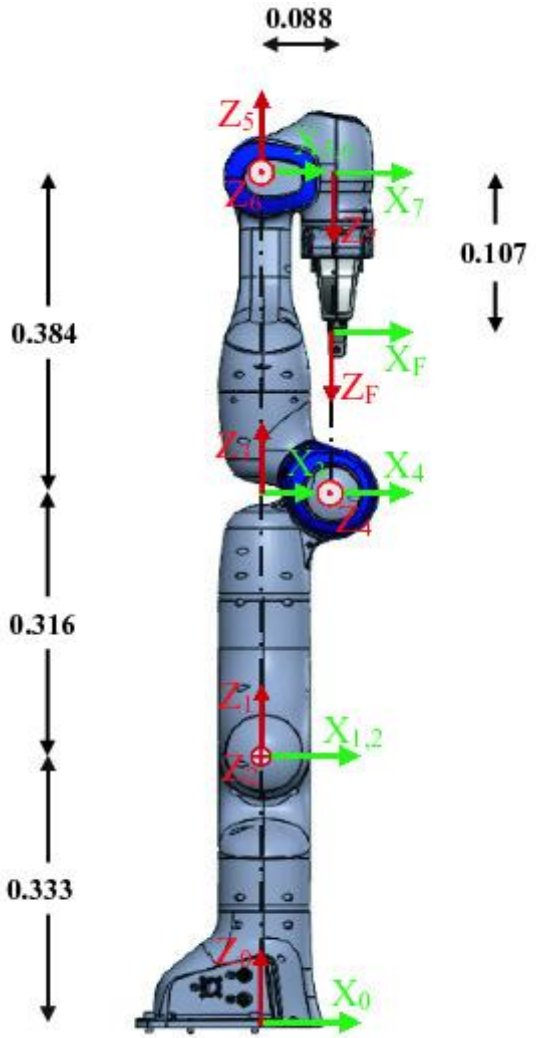
7 revolute joints











shorturl.at/cjv06

https://link.springer.com/chapter/10.1007/978-3-030-75259-0_11

Inverse Method for IK

- x^d the desired value of the end-effector pose
- Denote the forward kinematic map by $F(q)$
- $F(q)$ is a vector-valued function, each component gives one coordinate of the end-effector pose (e.g., position and orientation). If the end effector has m degrees of freedom and the robot has n joints,

$$x^d \sim F(q_1, \ldots, q_n) = \begin{bmatrix} f_1(\boldsymbol{q}) \\ \vdots \\ f_m(\boldsymbol{q}) \end{bmatrix}$$

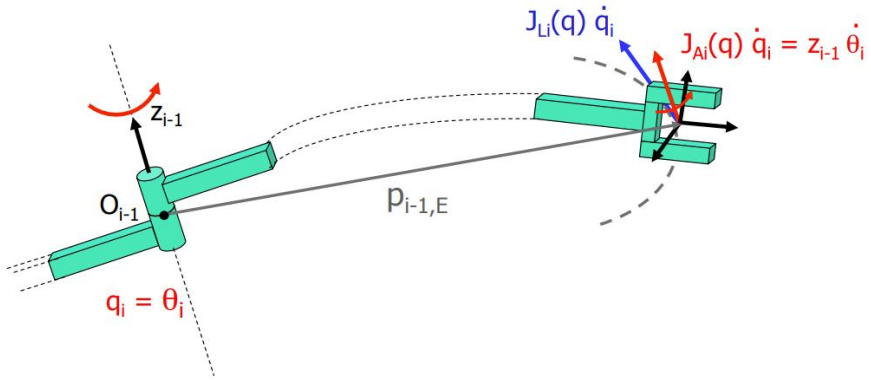
Iterative Method

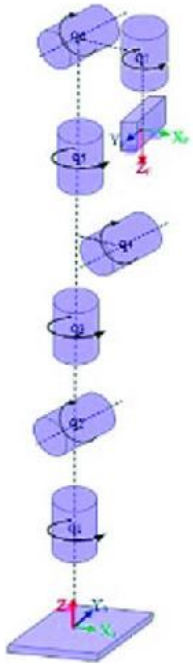
- Generate a sequence of values $\boldsymbol{q}^0$, $\boldsymbol{q}^1, \boldsymbol{q}^2 \ldots$ until we find some $\boldsymbol{q}^N$ such that $\boldsymbol{F}(\boldsymbol{q}^N)$ is sufficiently close to $\boldsymbol{x}^d$, i.e.,

$$\left\| \boldsymbol{F}(\boldsymbol{q}^N) - \boldsymbol{x}^d \right\| < \epsilon$$

- At each iteration, we compute $\boldsymbol{q}^{i+1}$ by

$$\boldsymbol{q}^{i+1} = \boldsymbol{q}^i + \Delta \boldsymbol{q}$$





Iterative Method

- What is a good value for $\Delta \boldsymbol{q}$?
- Gradient descent method
  - minimize the value of function $L(q) = \frac{1}{2}(F(q) - x^d)^T (F(q) - x^d)$
  - by choosing the updates according to $\Delta \boldsymbol{q} = -\nabla \mathbf{L}(\mathbf{q}^{\mathbf{i}})$ -- gradient w.r.t. $\boldsymbol{q}$
- For this problem, we define the iteration as

$$\boldsymbol{q}^{\boldsymbol{i}+\mathbf{1}} = \boldsymbol{q}^{\boldsymbol{i}} - \boldsymbol{\alpha}_{\boldsymbol{i}} \nabla \mathbf{L}(\mathbf{q}^{\mathbf{i}})$$

- We can use the scalar $\boldsymbol{\alpha}_{\boldsymbol{i}}$ to determine the magnitude of the step size
- But what is $\nabla \mathbf{L}(\mathbf{q}^{\mathbf{i}})$?

Calculate Error Gradient

$$L(q) = \frac{1}{2}(F(q) - x^d)^T (F(q) - x^d)$$

$$L(q) = \frac{1}{2}\begin{bmatrix}(f_1(q) - x_1^d) & (f_2(q) - x_2^d)\end{bmatrix}\begin{bmatrix}f_1(q) - x_1^d \\ f_2(q) - x_2^d\end{bmatrix}$$

$$= \frac{1}{2}(f_1(q) - x_1^d)^2 + \frac{1}{2}(f_2(q) - x_2^d)^2$$

$$\nabla L(q) = \begin{bmatrix}\frac{\partial L}{\partial q_1} \\ \frac{\partial L}{\partial q_2}\end{bmatrix}$$

$$\frac{\partial}{\partial q_1} L(q)$$

$$= \frac{\partial}{\partial q_1} \frac{1}{2}\left\{(f_1(q) - x_1^d)^2 + (f_2(q) - x_2^d)^2\right\}$$

$$\boxed{= \frac{\partial f_1}{\partial q_1}(f_1(q) - x_1^d) + \frac{\partial f_2}{\partial q_1}(f_2(q) - x_2^d)}$$

$$\frac{\partial}{\partial q_2} L(q)$$

$$= \frac{\partial}{\partial q_2} \frac{1}{2}\left\{(f_1(q) - x_1^d)^2 + (f_2(q) - x_2^d)^2\right\}$$

$$\boxed{= \frac{\partial f_1}{\partial q_2}(f_1(q) - x_1^d) + \frac{\partial f_2}{\partial q_2}(f_2(q) - x_2^d)}$$

Calculate Error Gradient

$$\nabla L(q) = \begin{bmatrix} \frac{\partial L}{\partial q_1} \\ \frac{\partial L}{\partial q_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial q_1}(f_1(q) - x_1^d) + \frac{\partial f_2}{\partial q_1}(f_2(q) - x_2^d) \\ \frac{\partial f_1}{\partial q_2}(f_1(q) - x_1^d) + \frac{\partial f_2}{\partial q_2}(f_2(q) - x_2^d) \end{bmatrix}$$

We can write this as a matrix equation

$$= \begin{bmatrix} \frac{\partial f_1}{\partial q_1} & \frac{\partial f_2}{\partial q_1} \\ \frac{\partial f_1}{\partial q_2} & \frac{\partial f_2}{\partial q_2} \end{bmatrix} \begin{bmatrix} f_1(q) - x_1^d \\ f_2(q) - x_2^d \end{bmatrix}$$

And we recognize that $\begin{bmatrix} \frac{\partial f_1}{\partial q_1} & \frac{\partial f_2}{\partial q_1} \\ \frac{\partial f_1}{\partial q_2} & \frac{\partial f_2}{\partial q_2} \end{bmatrix} = J^T(q)$, the arm Jacobian

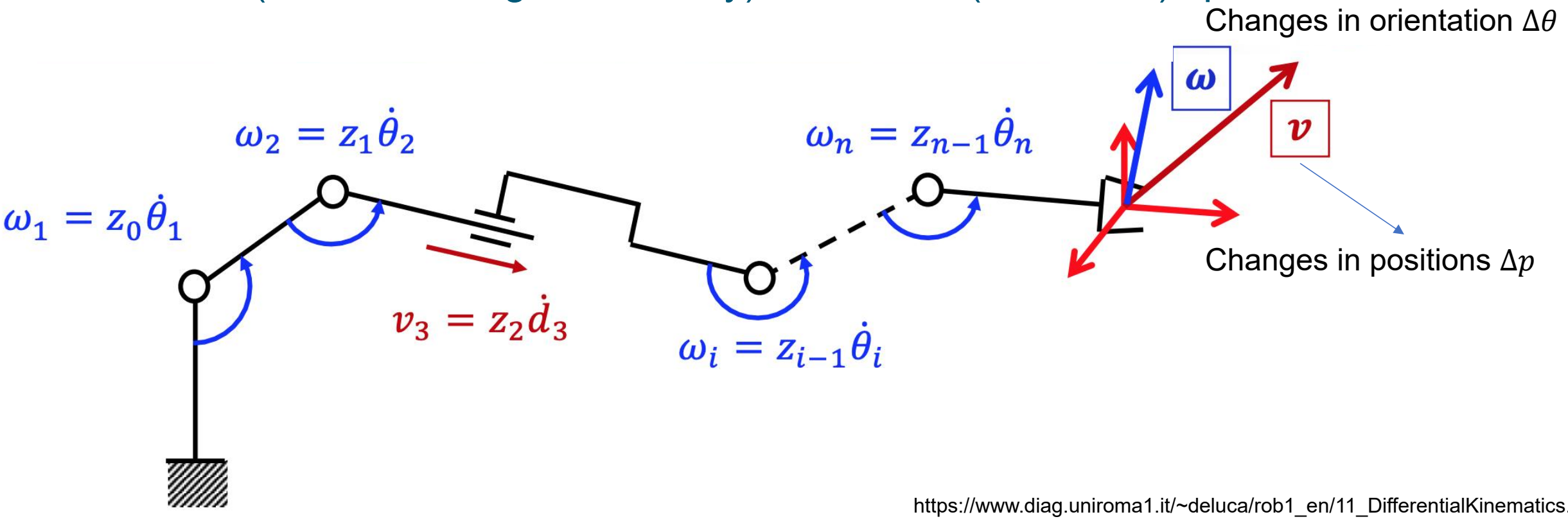
$$= \boldsymbol{J^T(q)}\,(\boldsymbol{f(q)} - \boldsymbol{x^d})$$

Jacobian Method

- Taylor series expansion for the forward kinematic map around the ith iteration
  - $F(q^i + \Delta q^i) = F(q^i) + J(q^i)\Delta q^i + h.o.t.$
  - *h.o.t.* refers to higher order terms (these go to zero quickly as$\Delta q^i \to 0$)
- If we neglect the higher order terms,Δq^i would be
  - $F(q^i + \Delta q^i) - F(q^i) \approx J(q^i)\Delta q^i$
  - $\Delta x^i \approx J(q^i)\Delta q^i$

Differential Kinematics

- Define the relationship between motion (velocity) in the joint space and motion (linear and angular velocity) in the task (Cartesian) space



https://www.diag.uniroma1.it/~deluca/rob1_en/11_DifferentialKinematics.pdf

Geometric Jacobian

- **geometric** Jacobian

$$\begin{pmatrix} v \\ \omega \end{pmatrix} = \begin{pmatrix} \dot{p} \\ \omega \end{pmatrix} = \begin{pmatrix} J_L(q) \\ J_A(q) \end{pmatrix} \dot{q} = J(q)\,\dot{q}$$

always a 6 x n matrix

$$\text{end-effector instantaneous velocity} \begin{pmatrix} v_E \\ \omega_E \end{pmatrix} = \begin{pmatrix} J_L(q) \\ J_A(q) \end{pmatrix} \dot{q} = \begin{pmatrix} J_{L1}(q) & \cdots & J_{Ln}(q) \\ J_{A1}(q) & \cdots & J_{An}(q) \end{pmatrix} \begin{pmatrix} \dot{q}_1 \\ \vdots \\ \dot{q}_n \end{pmatrix}$$

superposition of effects

$$v_E = \underbrace{J_{L1}(q)\,\dot{q}_1}_{\text{contribution to the linear e-e velocity due to } \dot{q}_1} + \ldots + J_{Ln}(q)\,\dot{q}_n \qquad \omega_E = \underbrace{J_{A1}(q)\,\dot{q}_1}_{\text{contribution to the angular e-e velocity due to } \dot{q}_1} + \ldots + J_{An}(q)\,\dot{q}_n$$

https://homes.cs.washington.edu/~todorov/courses/cseP590/05_Kinematics.pdf

Differential Kinematics: Geometric Jacobian

- Expression of geometric Jacobian (A **revolute** joint)

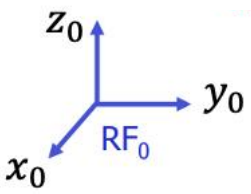
$$\left(\begin{bmatrix} \dot{p}_{0,E} \\ \omega_E \end{bmatrix} = \right) \begin{bmatrix} v_E \\ \omega_E \end{bmatrix} = \begin{bmatrix} J_L(q) \\ J_A(q) \end{bmatrix} \dot{q} = \begin{bmatrix} J_{L1}(q) & \cdots & J_{Ln}(q) \\ J_{A1}(q) & \cdots & J_{An}(q) \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \vdots \\ \dot{q}_n \end{bmatrix}$$

all vectors should be expressed in the same reference frame (here, the base frame RF_0)

	revolute i-th joint
$J_{Li}(q)$	$z_{i-1} \times p_{i-1,E}$
$J_{Ai}(q)$	z_{i-1}

this can be also computed as

$$= \frac{\partial p_{0,E}}{\partial q_i}$$

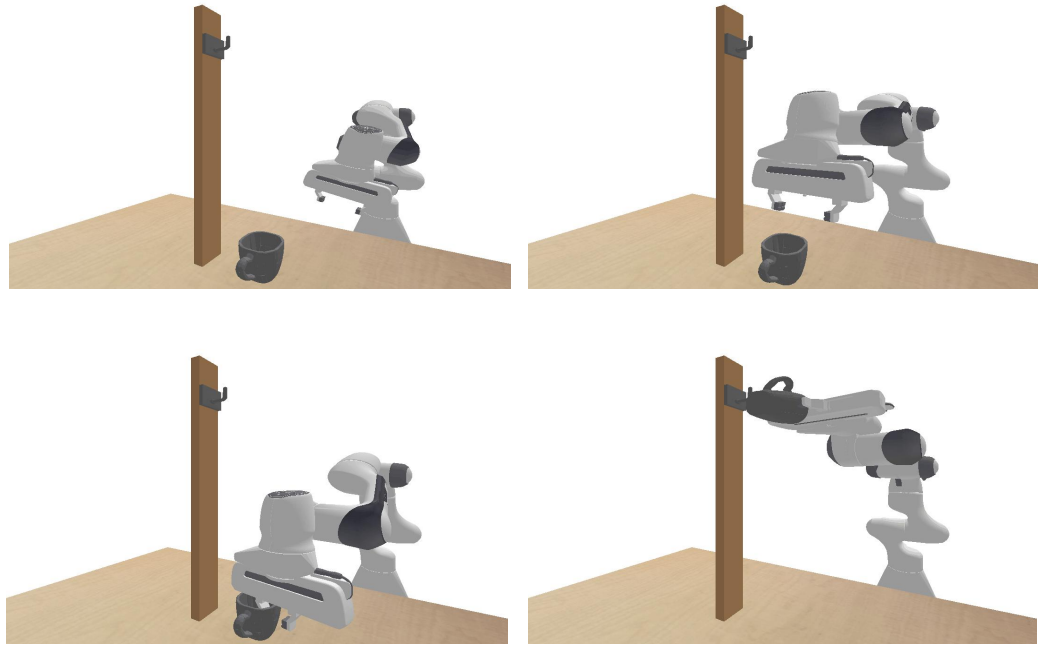


$$z_{i-1} = {}^0R_1(q_1)\ldots{}^{i-2}R_{i-1}(q_{i-1}) \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$p_{i-1,E} = p_{0,E}(q_1,\ldots,q_n) - p_{0,i-1}(q_1,\ldots,q_{i-1})$$

https://homes.cs.washington.edu/~todorov/courses/cseP590/05_Kinematics.pdf

The Concept Map for Robot Manipulation



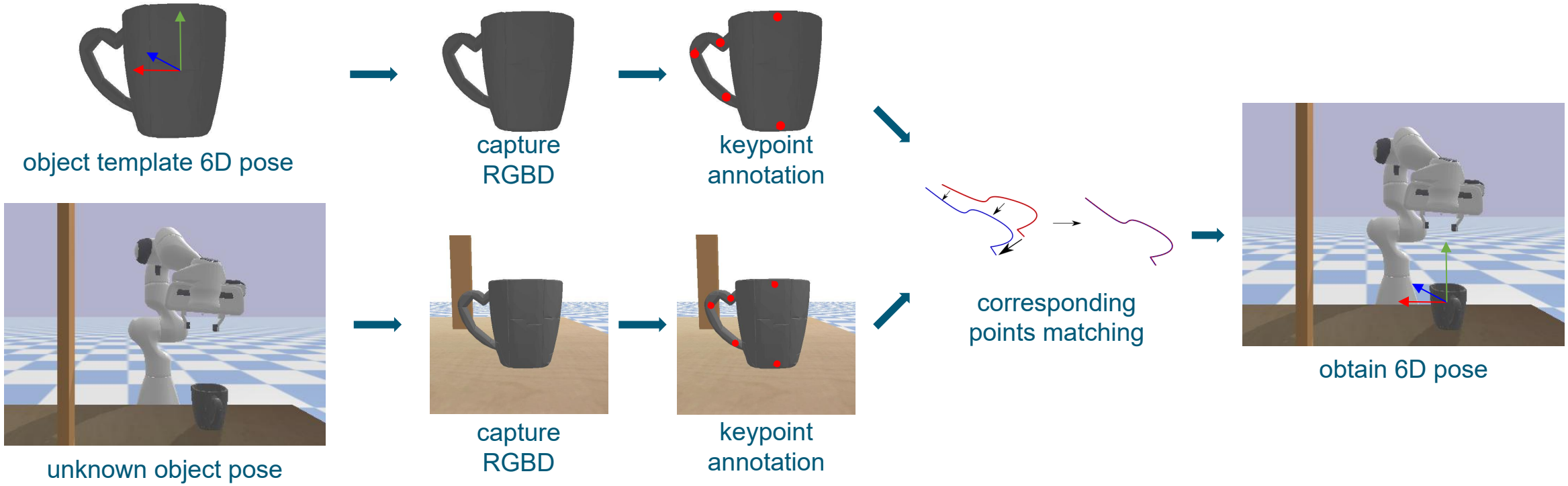
- Forward Kinematics
- 6D Pose Estimation
- Inverse Kinematics
- Grasping

6D Pose Estimation

- 6D pose is an important object representation in robot manipulation

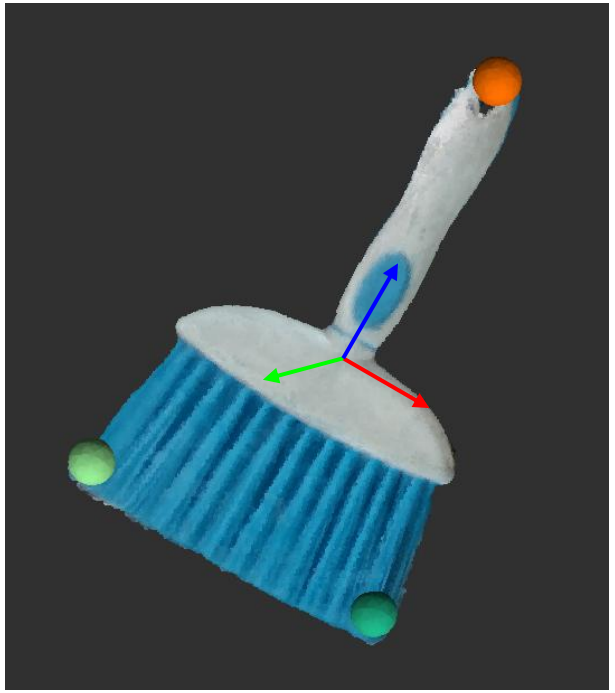
6D Pose Estimation: Template Pose Matching

- How to obtain the 6D pose of an object?
  - Template pose matching

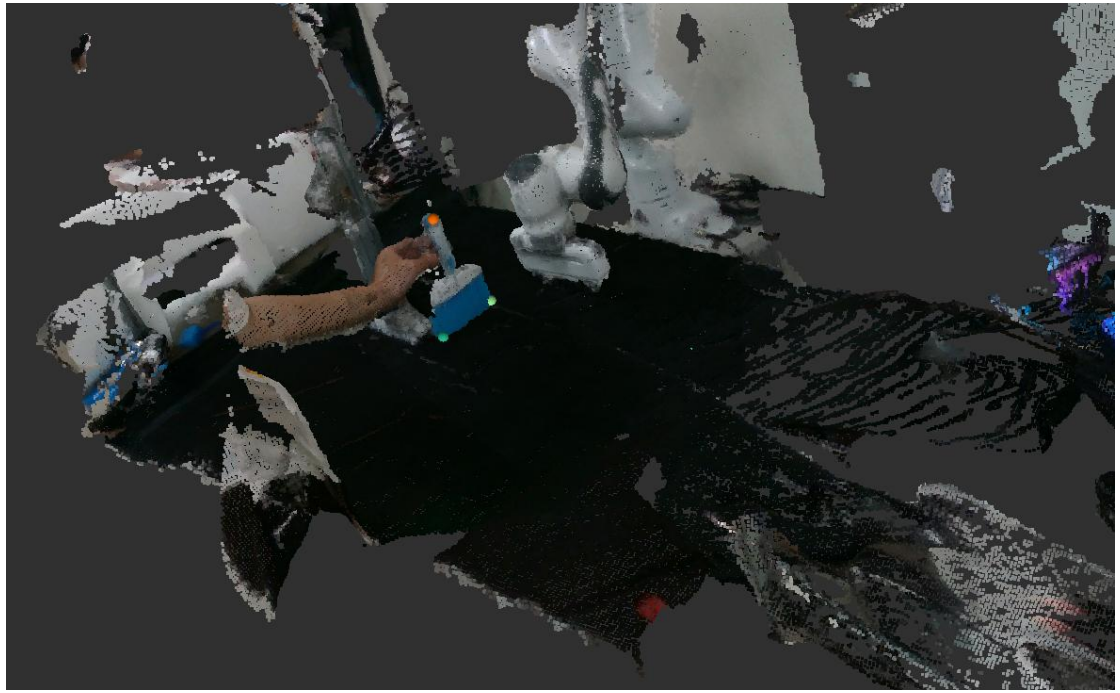


6D Pose Estimation: Template Pose Matching

An example in the real world:



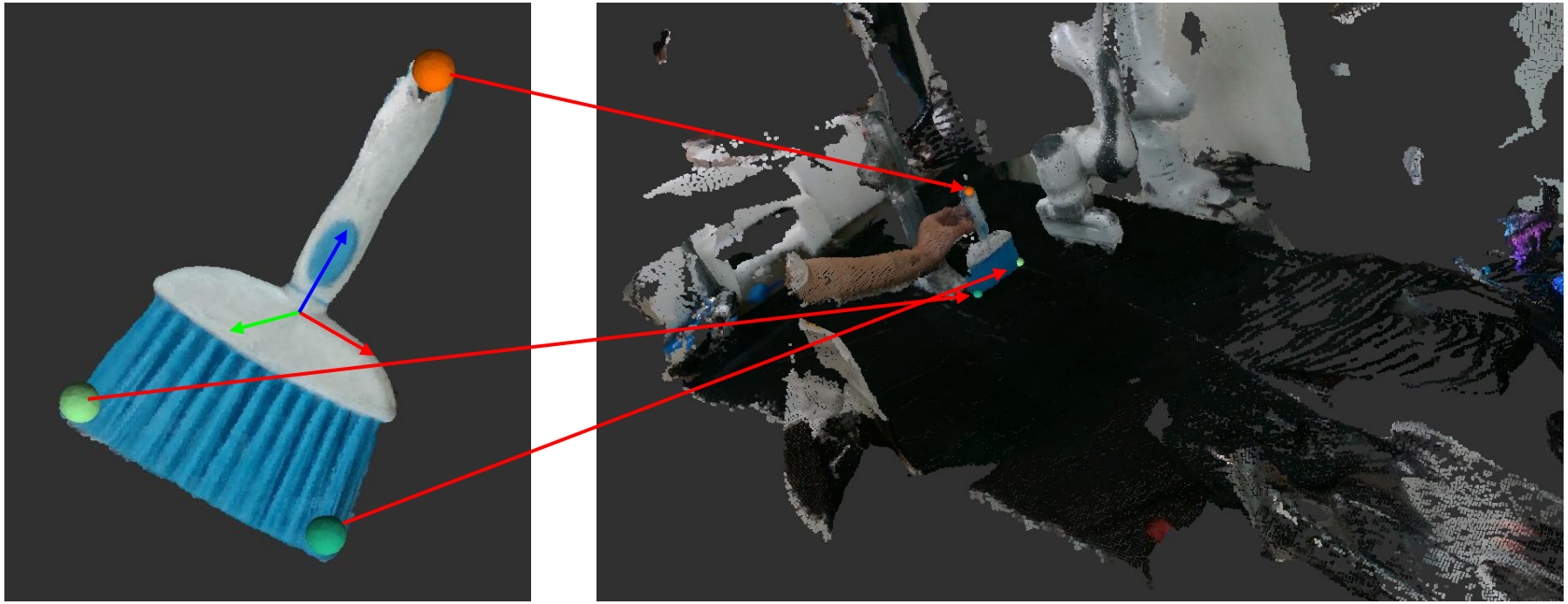
object template 6D pose



unknown object pose in a scene

6D Pose Estimation: Template Pose Matching

An example in the real world:

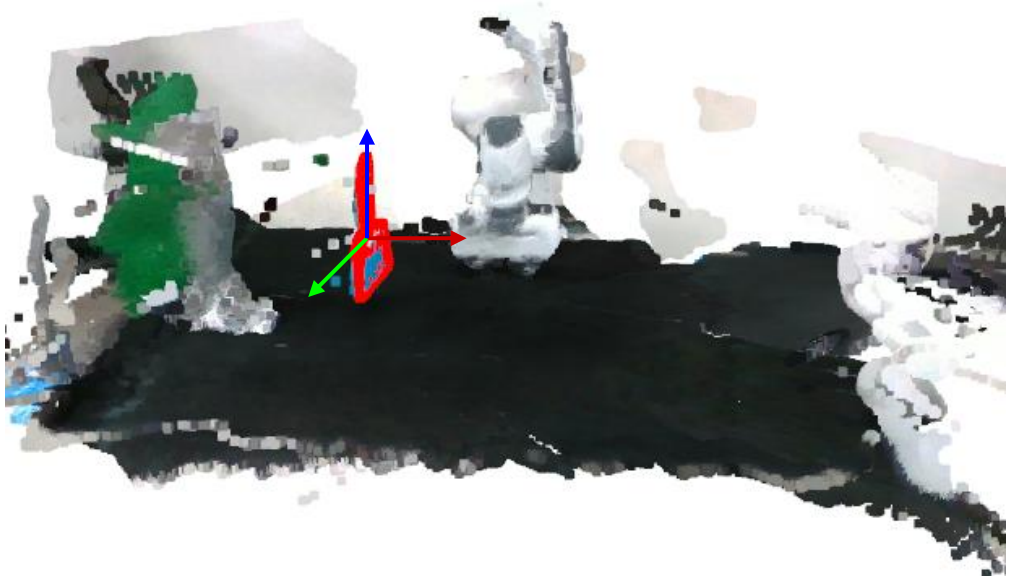


matching the object in the scene by using annotated keypoints

6D Pose Estimation: Template Pose Matching

An example in the real world:

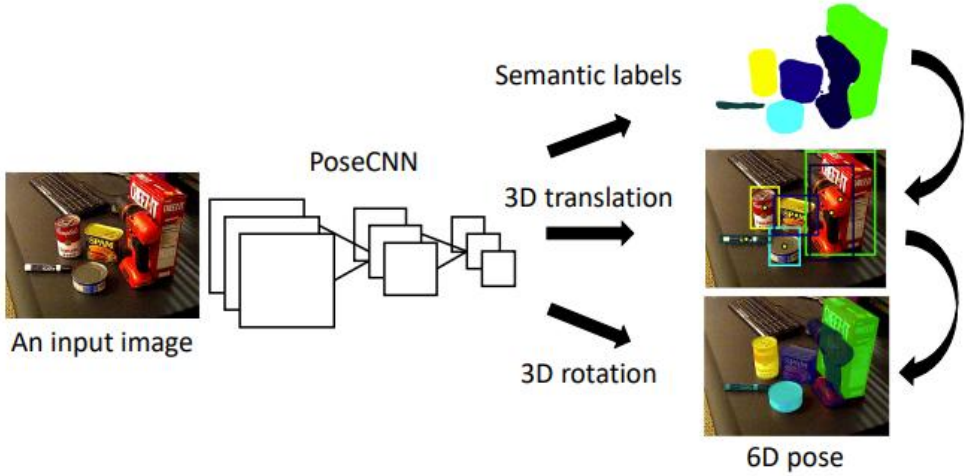
before matching



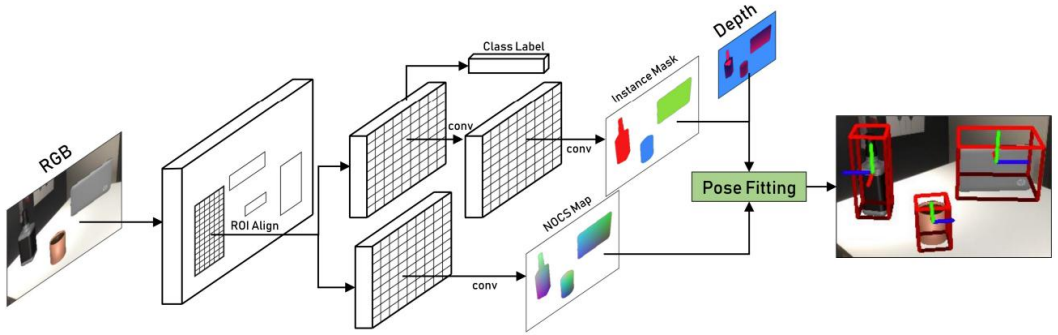
after matching

6D Pose Estimation: Other Approaches?

- How to obtain the 6D pose of an object?
  - Learning based methods



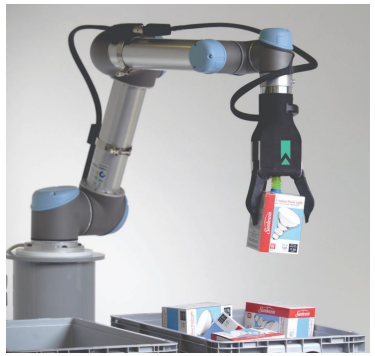
https://arxiv.org/pdf/1711.00199.pdf

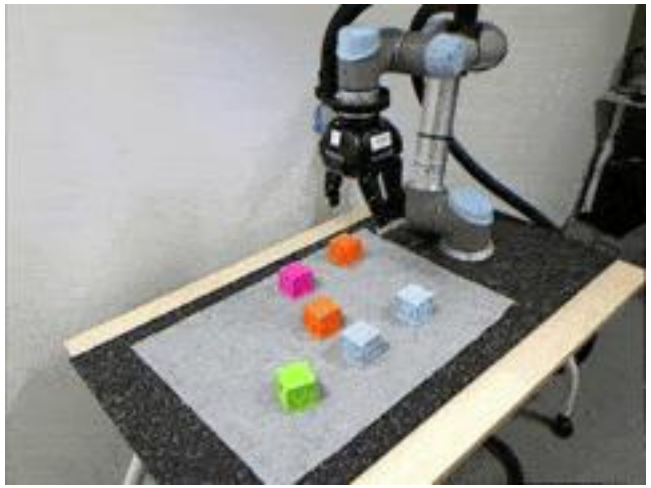


https://arxiv.org/pdf/1901.02970.pdf

Grasping

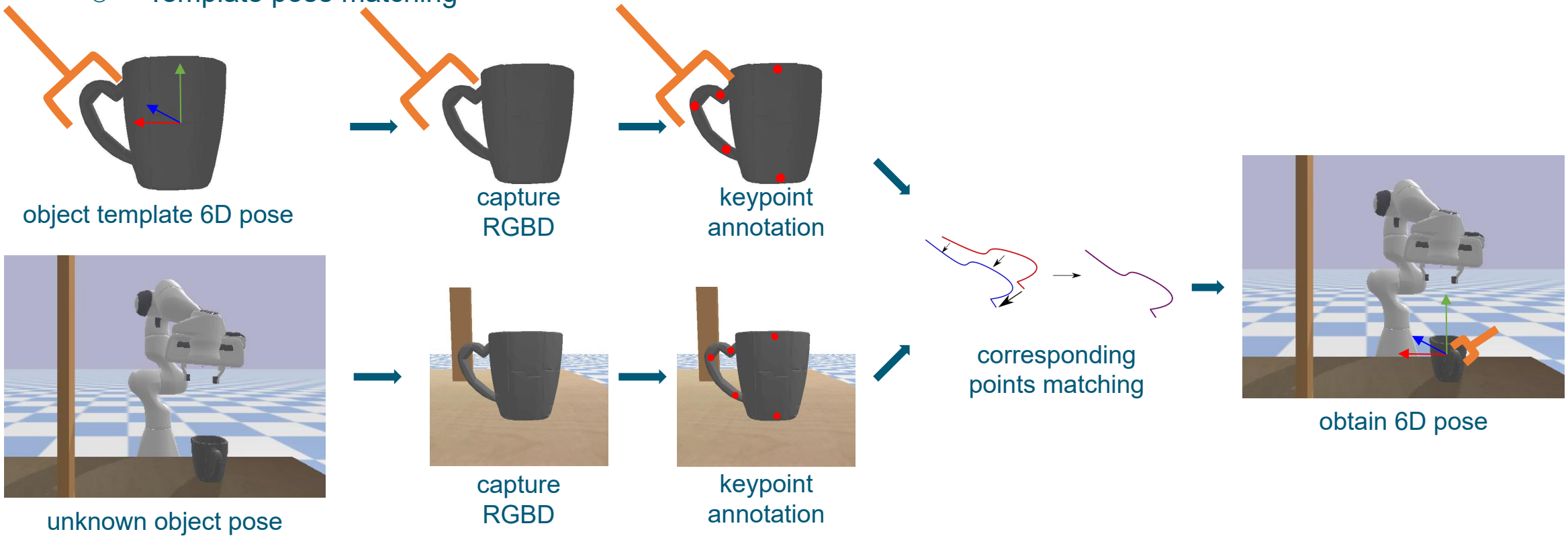
- Grasping is a very common action in robot manipulation
  - pick and place, rearrangement, tool use…





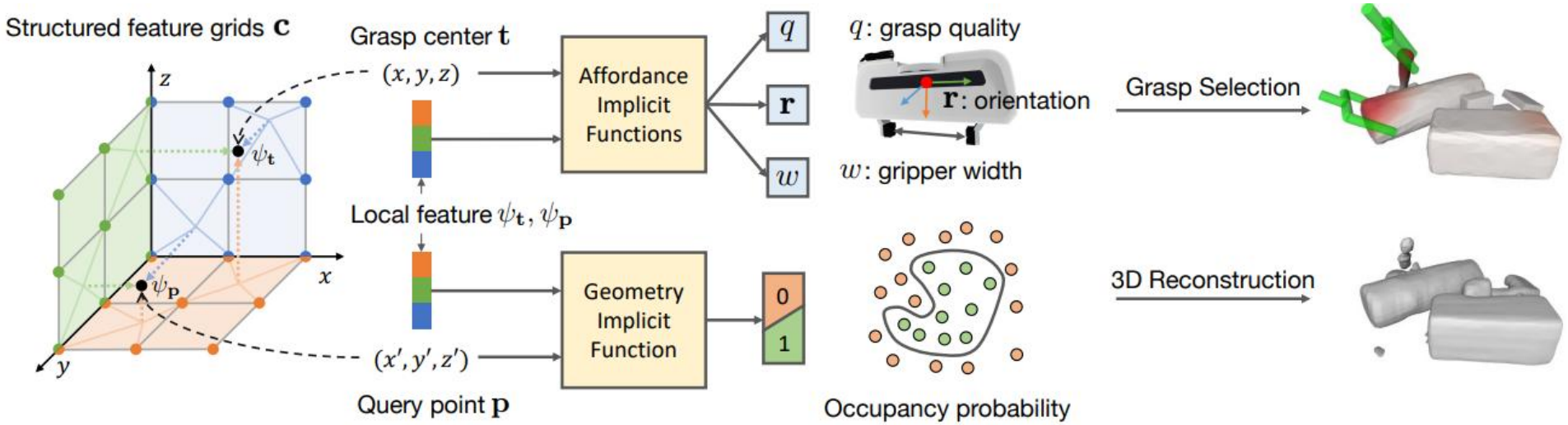
6D Pose Estimation: Template Pose Matching

- How to obtain the 6D pose of an object?
  - Template pose matching



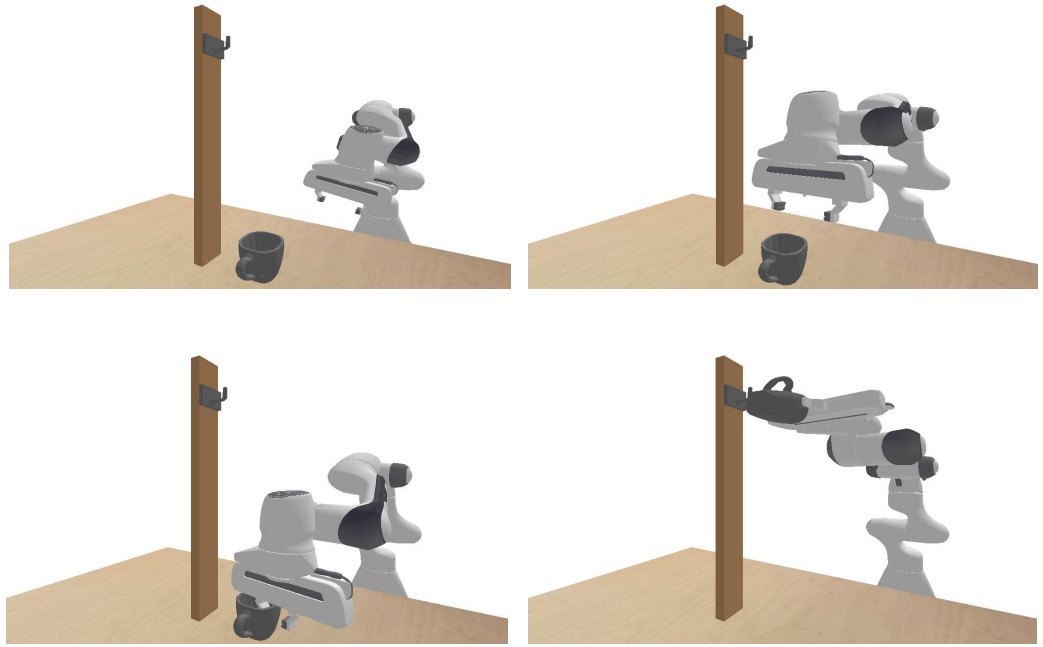
Grasping: other approaches

- How to obtain the 6D pose of an object?
  - Learning-based methods



https://arxiv.org/pdf/2104.01542v2.pdf

The Concept Map for Robot Manipulation



- Forward Kinematics
- 6D Pose Estimation
- Inverse Kinematics
- Grasping